

IntroToMC.odt
27jul24
24feb26

Beyond Arduino: Introduction to Industrial Micro-Controllers.
by Nels D. "Chip" Pearson (aka CmdrZin)

These tutorials will explore the Microchip tinyAVR 1-series of 8bit micro-controllers. Most, if not all, of the hardware resources of these devices will be demonstrated using multiple software examples and a variety of programming techniques. Numerous hardware interfacing examples to external devices will showcase the I/O capabilities of the devices.

The MPLAB X IDE system will be the main environment used for code development.

Target Devices:		FLASH	RAM	EEPROM	I/O	Price	
ATtiny212/412	8SIOC	2k/4k	128/256	64/128	6/6	\$0.49/25	\$0.53/25
ATtiny814/1614	14SIOC	8k/16k	512/2048	128/256	12/12	\$0.74/25	\$0.83/25
ATtiny1616/3216	20SIOC	16k/32k	2048/2048	256/256	18/18	\$1.05/25	\$1.17/25

Bare bones breakout boards for 8SIOC, 14SIOC, and 20SIOC device packages will be used with protoboards for demonstrations and experiments.

TODO:

~~Verify DAC section. Save as ITMC_III_3.X.zip.~~

~~Verify 6a. TCB section.~~

~~Verify SPI Host section.~~

~~Verify 7. WDT section.~~

~~Verify 8. RTC section.~~

~~Verify 9. CCL section.~~

~~Complete 4b. Needs schematic.~~

~~Verify SPI connections in 4a and 4b.~~

~~Add Figure III-10 schematic to Section III-10.~~

Add Figure 11 schematic to Section III-11.

Table of Contents

Introduction.....	4
SECTION I - Setup.....	5
1. Introduction and where to find additional information.....	5
a. Microchip's website.[1][3].....	5
b. UPDI Programmer.....	5
2. Establishing a Development Environment: Hardware and Software Integration.....	5
a. Hardware target.....	5
b. MPLAB X setup and UPDI programmers.....	6
c. Blink the LED.....	6
SECTION II - Exploration of Basic Hardware.....	7
1. Digital Input/Output (I/O).....	7
a. Pick a pin. Configure the PORT.....	7
b. Optional PORT Control.....	7
c. Output Options.....	7
A. Source and Sink current considerations.....	7
B. Control Logic.....	7
d. Input Options.....	8
A. Pull-ups considerations.....	8
2. Timing - Part 1 - Simple Timer - TCA.....	10
a. Configuring the TCA to count up and set a bit on overflow.....	10
3. Timing - Part 2 - Simple Timer - TCA using its interrupt.....	12
4. Timing - Part 3 - System Timer.....	14
5. General Programming.....	17
SECTION III - Exploration of Unique Hardware.....	19
1. Serial Communications - USART.....	19
a. Baud rate.....	20
b. Frame format.....	20
c. TxD as an output pin.....	20
d. Enable transmitter and receiver.....	20
2. Analog-to-Digital Converter - ADC.....	22
a. Setting up the ADC.....	22
3. Digital-to-Analog Converter - DAC.....	25
4. Serial Peripheral Interface – SPI.....	27
4a. SPI – Controller.....	27
4b. SPI - Peripheral.....	30
5. Two-Wire Interface - TWI (I2C).....	33
6. Other Timer/Counters.....	35
a. TCB.....	35
b. TCD.....	36
7. Watchdog Timer - WDT (and Sleep Controller - SLPCTRL).....	38
a. WDT setup.....	38
b. SLPCTRL setup.....	39
8. Real Time Counter - RTC.....	41
9. Configurable Custom Logic - CCL.....	42
10. Analog Comparator - AC.....	44
11. Brown Out Detector - BOD.....	46
12. Accessing EEPROM.....	47

.....	47
13. Accessing Flash Memory.....	48
SECTION IV - Example Projects.....	49
1. 20 LEDs controlled with 5 I/O lines.....	49
2. Remote Temperature Recorder.....	50
3. Interfacing Push Button Switches with debounce.....	50
4. Interfacing Single and Multiple 7-Segment LED Displays.....	50
5. HC-SR04 Ultra-sonic Interface for Distance Sensing.....	50
6. Make an LED Flicker like a Candle.....	50
7. Interface to a Small DC Motor though a H-Bridge Controller.....	50
8. Interface to Multiple RC Servos using TCA in Split Mode.....	50
9. Using the RTC for millis() to free up TCA.....	50
10. LCD Display interface for HD44780 Displays.....	52
11. IR Decoder with LCD Display.....	53
12. DDS Audio Signal Synthesis – DAC and PWM.....	54
SECTION V - Power.....	56
APPENDIX I - BOARD LAYOUTS.....	57
A. ATtiny8SIOC.....	57
B. ATtiny14SIOC.....	58
C. ATtiny20SIOC.....	59
APPENDIX III – Schematics.....	60
A. ATtiny8SIOC.....	60
B. ATtiny14SIOC.....	61
C. ATtiny20SIOC.....	62
APPENDIX IV - UPDI Programmers.....	63
References.....	63

Introduction

This project assumes an average level of experience with the Arduino development environment. Its goal is to free you from the constraints of the Arduino 'pre-determined hardware use' environment and allow the hardware resources of the micro-controller to be used as you see fit. Code examples will show the user how to replicate many of the Arduino support functions without being limited to the ATmega328P device used on most Arduino boards.

The Microchip tinyAVR 1-series family was selected due to its straight forward architecture, low cost, and variety of hardware resources in such small packages. Although these devices are SIOC packages (break-out boards are available), similar Atmel devices from the prior generation are still available in DIP packages and can be substituted with minimal effort.

The processes followed in this document to explore this family of Microchip devices can be applied to the newer AVR EA, EB, DA, DB, and DU families as well. Use the link[1] to obtain more information.

Many code examples use macros for bit settings. These usually end in `_bp`, `_bm`, or `_gc`. The `_gc` is a group code macro and represents a group of bits positioned for a specific area in the register. For example, `ADC_RESSEL_8BIT_gc` is a pattern 00000100 which was generated by shifting a 01 pattern two bits to the left with a `(0x01<<2)` statement and is used to set the `RESSEL` area of the `ADCN.CTRLA` register. (see `iotn412.h` for more examples). The `_bm` is a bit mask for an individual bit at a specific position. Its position is generated with a bit shift statement like `(1<<3)` which will generate the pattern 00001000. For example if `A=(1<<3)`, then to set the bit, use

```
reg |= A;
```

to OR the bit with the reg contents and to clear that bit position, use

```
reg &= ~A;
```

which will AND the inverse of the pattern (11110111) with the reg contents. The `_bp` is just the number of the bit position. In the prior example, if we used `#define X_bp 3`, then we could use `A=(1<<X_bp)` to get the same results. X could be replaced with a meaningful name to make the code more readable.

SECTION I - Setup

1. Introduction and where to find additional information.

a. Microchip's website.[1][3]

Use the link[3] to access the Microchip website, download the MPLAB X IDE, and install it. The IDE is available for Windows, Linux, and macOS systems. Be sure to install the XC8 compiler.

b. UPDI Programmer

You will also need a UPDI capable programmer to program/debug the target device. (see the **UPDI Programmers** section for suggestions)

2. Establishing a Development Environment: Hardware and Software Integration.

a. Hardware target.

Simple solder-less breadboard or protoboard with a DC power supply between 3.0 and 5.0 vdc able to provide a minimum of 100ma of current.

A three cell AA or AAA battery pack (4.5v with standard batteries, 3.6v with rechargeable) works well as would a LiPo battery (3.7v). The devices can use supply voltages from 1.8v to 5.5v.[2] The target MCU will be an 8SIOC device mounted on a breakout board.

Figure 1 shows how the power supply, MCU board, and SNAP UPDI header are wired together.

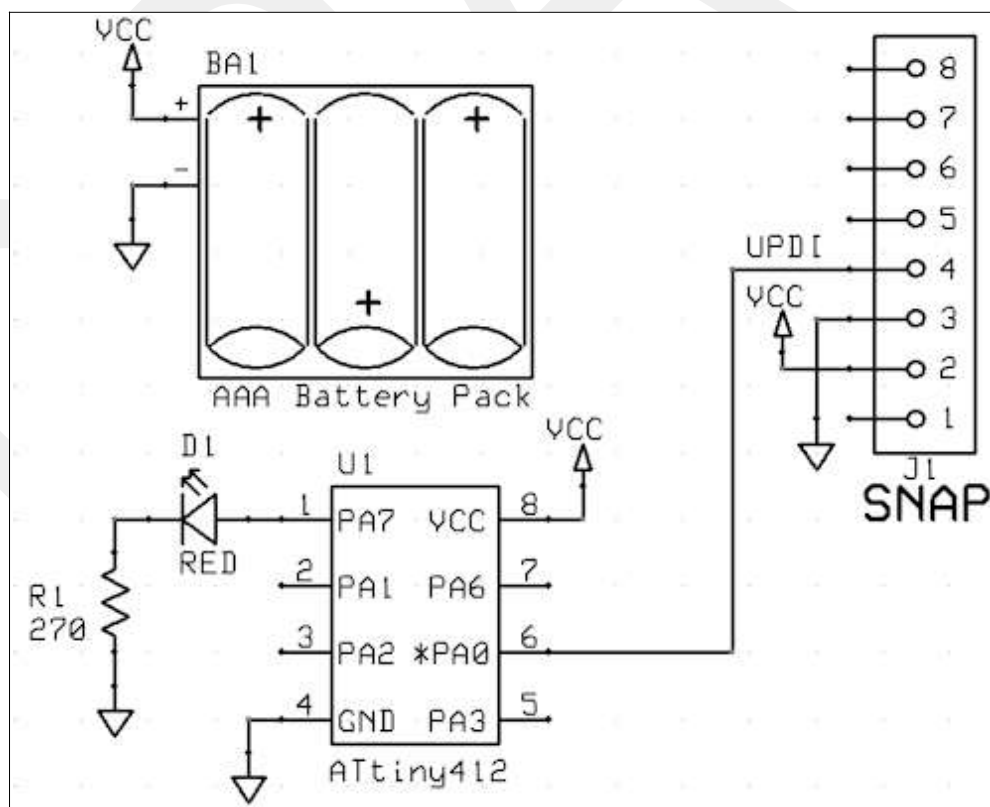


Figure I-1

b. MPLAB X setup and UPDI programmers.

Download and install MPLAB X IDE from the Microchip website.[3]

Open MPLAB and connect the SNAP module to the computer.

Select File > New Project Microchip Embedded > Standalone Project then Next >.

In the Select Device pop-up, use

Family: 8-bit AVR MCUs (XMega/Mega/Tiny/AVR)

Device: ATtiny412

Tool: SNAP or Simulator if you do not have a programmer tool.

Next >

In the Select Compiler, pick XC8. Next >

Select a Project Name and Folder to store the project into. Finish.

Once the project is set up, RMC "Source File" in the Projects list then New > avr-main.c

Change File Name to main. Finish.

NOTE: The SNAP programmer has to be modified per ETN #36 "MPLAB SNAP AVR UPDI/PDI/TPI Interface Modification engineering technical note". It may also need to have its software updated using the MPLAB IPE. This is evident if the connection window shows no S/N for the SNAP when trying to connect.

If another programmer is used, use the MPLAB IPE and the hex file from the ../dist/default/production directory in the project folder.

c. Blink the LED.

Replace the main(){ } code in the main.c file with this code.

```
int main(void)
{
    uint16_t count = 0;
    /* set PA7 of PORT A as an output pin */
    PORTA.DIR |= PIN7_bm;

    // Toggle pin every 65536 counts.
    while (1) {
        if( count == 0) {
            PORTA_OUTTGL = PIN7_bm;
        }
        ++count;
    }
}
```

Compile the code and download it into the device.

The complete project, ITMC_I_2.X.zip, can be downloaded from "<https://www.gameactive.org/dist/>".

SECTION II - Exploration of Basic Hardware

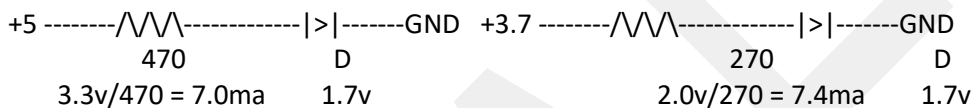
1. Digital Input/Output (I/O)

Now for an explanation of what it took to control the LED. A typical red LED requires about 1ma of current flowing through it to generate detectable light. Almost all of them are rated for a maximum of 20ma continuous current and will tolerate short pulses of 40ma or more.

A little electronics theory will explain this common LED circuit seen in many examples. It consists of a resistor in series with an LED and is designed to result in a nominal 5ma to 10ma to light up the LED. The resistor has to be adjusted based on the power source.

Two circuits are shown; one using a 5v source (typical of many micro-controllers) and another using a 3.7v source (a system using a LiPo battery as its power source). A typical RED LED has a voltage drop of about 1.7v. The resistor (R) has to cause a voltage (E) drop due to the current (I) such that the total of voltage changes in the circuit is zero.

Using Ohm's law, $E = I * R$, or $E/R = I$ to calculate the resistor value for a nominal current value, we have.



So, $+5v - 3.3v - 1.7v = 0$ and $+3.7v - 2.0v - 1.7v = 0$.

a. Pick a pin. Configure the PORT.

Now that the external circuit parameters are known, 5v @ 7.0ma or 3.7v @ 7.4ma, to light the LED, you need to determine if this micro-controller can meet these requirements.

We will use the ATtiny412 device and a 3.6 AAA battery pack with NiCad rechargeable batteries and PA7 as the control pin.

Section 35.10 Table 35-16[2] states the I/O Pin Characteristics. V_{OH} gives the I/O pin drive strength with $V_{DD} = 3.0v$ and an output current of 7.5ma, the pin will maintain a minimum level of 2.4v. So for a V_{DD} of 3.6v, it should maintain about 3.2v or so which is still high enough to light the LED.

Note that higher currents are available with higher V_{DD} supply voltages. i.e. use +5v V_{DD} and a smaller resistor (allowing more current to flow) for brighter LEDs!

b. Optional PORT Control

Section 16.3.2.2[2] states that the Virtual PORT register map method should be used to control I/O port. This provides some benefits, as explained, but we will use the standard name PORT. Table 16-1[2] shows that to access the regular port registers, just use PORT instead of VPORT.

c. Output Options.

A. Source and Sink current considerations.

The example schematic in Figure 1 is using the MCU pin to source current to the LED. The current flows out of the pin, through the LED and resistor, and into ground (the negative side of the battery).

The LED could have been connected to VCC and then the MCU pin would act as a current sink.

The current would flow from the battery plus terminal, through the LED and resistor, and into the MCU pin.

Whether the circuit is configured for a source or a sink, the calculations for the resistor still have to result in the total voltage changes being zero and the MCU pin has to be able to handle the source current or the sink current. Always check the data sheet! Some MCUs can sink more current than they can source.

B. Control Logic

To use an I/O PORT pin as an output, source or sink, the bit in the direction register PORTA.DIR associated to the pin must be set to a '1'. The example code line

```
PORTA.DIR |= PIN7_bm;
```

does this. The 'x' in PORTx has been replaced with the port being used; an 'A' for this device. Some devices have many ports: PORTA, PORTB, PORTC, etc. Just use the appropriate letter for the port to be controlled.

The PIN7_bm is a defined literal for the bit pattern 1000 0000. Looking at 16.5.1[2], the DIR register, this pattern OR'd into the register will 'set' bit 7 to a '1' and cause PA7 to be an output pin with its output driver enabled.

The output pin can be set HIGH or LOW by writing a '1' or a '0' respectively using the

```
PORTA.OUT = <pattern>
```

command. In the example code to toggle the bit, the

```
PORTA_OUTTGL = PIN7_bm;
```

command is used with the bit pattern for pin 7.

Other control registers are discussed in section 16.5.1[2] and you may want to experiment with them.

d. Input Options.

A. Pull-ups considerations.

The example schematic in Figure II-1 has added a push-button switch to circuit from Figure I-1.

This is a common configuration used for simple user inputs. Note that it does not have an external pull-up resistor to pull the line HIGH when the button is not pressed. To minimize part count, most MCUs have optional internal pull-up resistors. Section 36.10[2] shows this internal resistor to be about 35k, when activated.

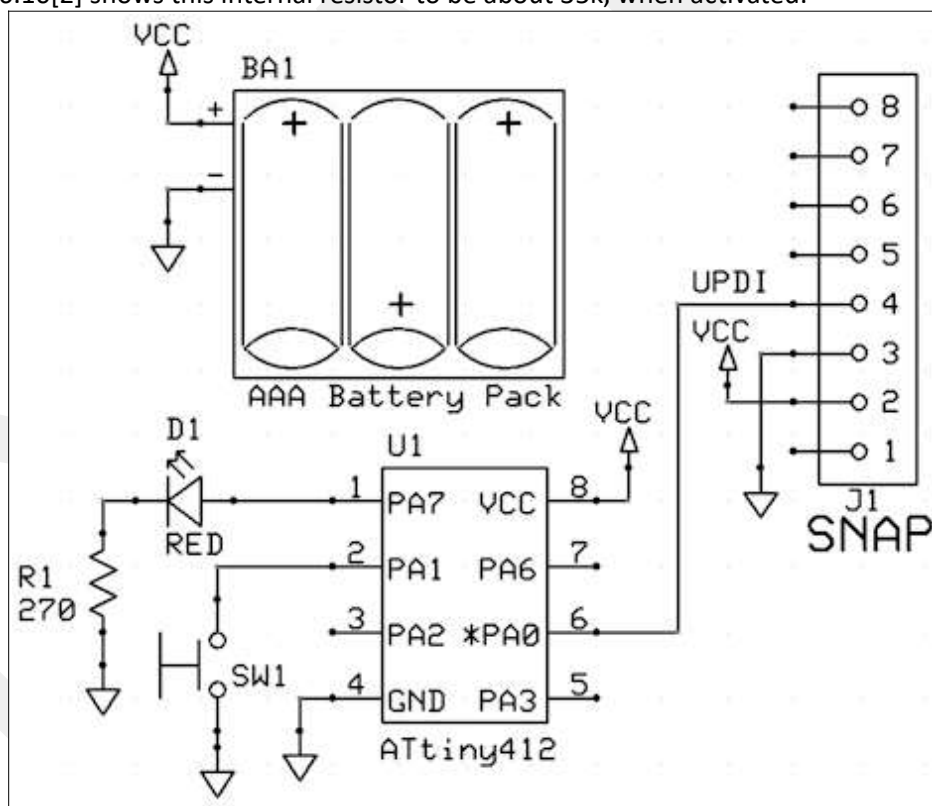


Figure II-1

To use an I/O PORT pin as an input, the bit in the direction register PORTA.DIR associated to the pin must be set to a '0'. Since this is the reset condition of the bit (see section 16.5.1[2], nothing needs to be done. If you need to clear this bit due to other code operations, this example line of code could be used.

```
PORTA.DIR &= ~PIN1_bm;
```

It inverts the bit mask 0000 0010 to 1111 1101 and ANDs it to the DIR register contents.

Another register, PORTA.PINnCTRL (see 16.5.11 [2]) also affects the operation of this pin. The 'n' refers to the pin number, so 'n=1' in this case.

The reset (default) settings for all pins is Non-Inverted, Pull-up disabled, and INTDISABLE.

Since the pull-up for pin 1 will be used, set bit 3 (PULLUPEN) in the register to '1' with

```
PORTA.PIN1CTRL |= (1<<3); // set PULLUPEN = 1
```

Read the state of the pins into a variable. For example, use

```
portA_pins = PORTA.IN; // read all the pins of PORTA.
```

To test this new configuration, replace the main(void) code (main02.c) with the following:

```
int main(void) {
    uint8_t portA_pins = 0;

    /* set PA7 of PORT A as an OUTPUT pin. The other bits are left as '0'
     * so that their associated pins will be INPUT pins. */
    PORTA.DIR |= PIN7_bm;

    /* enable the internal Pull-up resistor for PORTA pin 1 */
    PORTA.PIN1CTRL |= (1<<3); // shift the '1' into the bit3 position and OR it.

    // Control the LED with the push button switch.
    while (1) {
        portA_pins = PORTA.IN; // read all the pins of PORTA.
        portA_pins &= PIN1_bm; // set all bits except bit.1 to '0'.
        if( portA_pins == 0 ) {
            /* turn ON the LED if bit.1 (pin1) is LOW (i.e. switch is closed) */
            PORTA_OUTSET = PIN7_bm;
        } else {
            /* turn OFF the LED if pin1 is HIGH (i.e. switch is open) */
            PORTA_OUTCLR = PIN7_bm;
        }
    }
}
```

Compile the code and download it into the device. The LED can now be controlled by the switch.

The complete project, ITMC_II_1234.X.zip, can be downloaded from " <https://www.gameactive.org/dist/> ".

Now that basic INPUT and OUTPUT control is available, try adding more LEDs and switches to do more complex operations. A simple system might use two LEDs, one RED and one GREEN, and three switches; A, B, and C. When a majority of switches are pressed the GREEN LED is turned ON. If not, the RED LED is lit instead.

2. Timing - Part 1 - Simple Timer - TCA

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3217-Getting-Started-with-TCA-DS90003217.pdf>

Section 10[2] describes the clock control registers. On reset, these registers default to generating a 3.3 MHz peripheral clock (CLK_PER) that is used as the base clock for all peripherals. This will be the starting point for exploring Timer A; a 16-bit Timer/Counter described in Section 20[2]. The goal is to blink the LED in Figure 1 at a constant rate.

a. Configuring the TCA to count up and set a bit on overflow.

The TCA has many features and operating mode. The ref above explores them and provides example code to experiment with. This section will look at the most basic of timer operations: counting internal clock pulses (CLK_PER).

The counter will be used in the NORMAL mode (see 20.3.3.1[2]). This is a 16-bit counter and will overflow back to 0 every 65536 counts. The CLK_PER is CLK_MAIN (usually 20MHz) divided by 6 resulting in a 3.3 MHz (3.333333×10^6) clock which would cause the counter to overflow every

$$20 \times 10^6 / 6 / 65536 = 51.0 \text{ Hz}$$

Too fast to see a blinking LED. Setting the TCA.CTRLA to divide the incoming CLK_PER by 64 will slow the overflow rate down to about 0.8 seconds.

The counter also must be enabled to operate. The code below can be used to do these tasks.

```
/* set Normal mode */
TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_NORMAL_gc;

TCA0.SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV64_gc /* set CLK_PER/64) */
| TCA_SINGLE_ENABLE_bm; /* start timer */
```

When it overflows, it will set the OVF bit in the INTFLAGS register (see 20.5.11[2]). This bit is not cleared automatically and must be cleared by writing a '1' to its bit location to allow it to be set on the next overflow. The following example code can be used to do this.

```
/* The OVF flag has to be cleared manually */
TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;
```

The following example code (main03.c) will use this information to toggle the LED in Figure 1 on or off approximately every 1.3 seconds.

```
int main(void) {
    /* set PA7 of PORT A as an OUTPUT pin. The other bits are left as '0'
     * so that their associated pins will be INPUT pins. */
    PORTA.DIR |= PIN7_bm;

    /* *** TCA Configuration as NORMAL counter *** */
    /* set Normal mode */
    TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_NORMAL_gc;

    TCA0.SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV64_gc /* set CLK_PER/64) */
    | TCA_SINGLE_ENABLE_bm; /* start timer */

    // Blink the LED at a rate set by the TCA overflow rate.
    while (1) {
        /* Test for overflow */
        if( TCA0.SINGLE.INTFLAGS & TCA_SINGLE_OVF_bm) {
            /* reset the flag */
            /* The OVF flag has to be cleared manually */

```

```

        TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;
        /* and toggle the LED state. */
        PORTA_OUTTGL = PIN7_bm;
    }
}

```

Compile the code and download it into the device. The LED is now being controlled by TCA timer.

This modified code is in the main03.c file. The complete project, ITMC_II_1234.X.zip, can be downloaded from "<https://www.gameactive.org/dist/>".

3. Timing - Part 2 - Simple Timer - TCA using its interrupt

Configuring the TCA for NORMAL operation is the same as in the previous section. The additional step is to enable the interrupt on overflow bit in register TCA0 . INTCTRL. The following code can do that.

```
/* enable overflow interrupt */
TCA0.SINGLE.INTCTRL = TCA_SINGLE_OVF_bm;
```

Since an interrupt is being active, it must have an interrupt handler (ISR) to take control when the MCU jumps to the interrupts vector. The following example code will do that. This code will also toggle the LED.

```
ISR(TCA0_OVF_vect)
{
    /* toggle the LED state. */
    PORTA_OUTTGL = PIN7_bm;
    /* The interrupt flag has to be cleared manually */
    TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;
}
```

Additionally, the interrupt.h header is needed and the command to enable global interrupts. This is the modified main.c file (main04.c) code for this example. The changes are in **BOLD**.

```
#include <avr/io.h>
#include <avr/interrupt.h>           // to support the use of interrupts

/* TCA interrupt service routine. */
ISR(TCA0_OVF_vect)
{
    /* toggle the LED state. */
    PORTA_OUTTGL = PIN7_bm;
    /* The interrupt flag has to be cleared manually */
    TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;
}

int main(void) {
    /* set PA7 of PORT A as an OUTPUT pin. The other bits are left as '0'
    * so that their associated pins will be INPUT pins. */
    PORTA.DIR |= PIN7_bm;

    /* *** TCA Configuration as NORMAL counter with interrupt *** */
    /* enable overflow interrupt */
    TCA0.SINGLE.INTCTRL = TCA_SINGLE_OVF_bm;

    /* set Normal mode */
    TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_NORMAL_gc;

    TCA0.SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV64_gc /* set CLK_PER/64) */
                      | TCA_SINGLE_ENABLE_bm;    /* start timer */

    /* enable Global interrupts */
    sei();

    // Blink the LED at a rate set by the TCA overflow rate.
    while (1) {
        ;           // LED control is handled by the interrupt service.
    }
}
```

Compile the code and download it into the device. The LED is now being controlled by TCA timer interrupt service routine.

As a simple test, change `TCA_SINGLE_CLKSEL_DIV64_gc` to `TCA_SINGLE_CLKSEL_DIV16_gc` to make the LED flash 4 times faster.

This modified code is in the `main04.c` file. Replace `main.c` or rename to `main.c`. The complete project, `ITMC_II_1234.X.zip`, can be downloaded from " <https://www.gameactive.org/dist/> ".

4. Timing - Part 3 - System Timer

A very common resource in many systems is the one millisecond time reference or `millis()` function. This function returns a value equal to the number of milliseconds that the system has been operating since reset. The value is normally an unsigned long or `uint32_t` and will overflow about every 50 days so, long enough for most projects.

TCA will be configured to generate an interrupt every 1ms and additional code will be added to provide the `millis()` function. The `millis()` function will then be used to control the LED in Figure 1.

The first task is to configure the TCA to interrupt every 1ms. Since the `CLK_PER` is at 3,333,333 Hz and if the prescaler is set to x1, the counter will be at 1ms after 3,333 counts. The `TCA0_SINGLE_PER` register sets the TOP value for the counter. This is the overflow point. At reset, this TOP value is 0xFFFF, so the full 16-bits of the counter are used. The configuration code for the TCA is now:

```
/* *** TCA Configuration as NORMAL counter with interrupt *** */
TCA0_SINGLE_PER = 0x0D04;    // 3,333 - 1 since the counter starts at 0.
TCA0_SINGLE.INTCTRL = TCA_SINGLE_OVF_bm;
TCA0_SINGLE.CTRLB = TCA_SINGLE_WGMODE_NORMAL_gc;
TCA0_SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV1_gc /* set CLK_PER/1) */
                  | TCA_SINGLE_ENABLE_bm;    /* start timer */
```

The ISR will now update a global variable to accumulate 1m 'ticks'. A service function will allow other parts of the program to access the accumulated value in a reliable method. This is the additional code.

```
volatile static uint32_t totalMilliseconds;    // global variable

/* This interrupt service is called every 1 ms. */
ISR(TCA0_OVF_vect)
{
    ++totalMilliseconds;
    TCA0_SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm; // clear interrupt flag.
}

/* Return the total number of milliseconds since the project started. */
uint32_t millis()
{
    uint32_t temp;           // make a holder for the counter.
    uint8_t sreg_save;

    // Save SREG, disable interrupts, do code, restore SREG.
    // If interrupts were enabled, they will be re-enabled.
    sreg_save = SREG;
    cli();           // Turn OFF interrupts to avoid corruption
                    // during a multi-byte read.
    temp = totalMilliseconds; // get a copy while interrupts are disabled.
    SREG = sreg_save;

    return temp;       // return a 'clean' copy of the counter.
}
```

The new `main.c` file (`main05.c`) should now look like this.

```
#include <avr/io.h>
#include <avr/interrupt.h>    // to support the use of interrupts

#define LED_DELAY    1000UL    // N * 1ms

volatile static uint32_t totalMilliseconds;    // global variable
```

```

uint32_t millis();                // prototype for function

/* TCA interrupt service routine. */
ISR(TCA0_OVF_vect)
{
    ++totalMilliseconds;
    TCA0.SINGLE.INTFLAGS = TCA_SINGLE_OVF_bm;    // clear the interrupt flag
}

/* Return the total number of milliseconds since the project started. */
uint32_t millis()
{
    uint32_t temp;                // make a holder for the counter.
    uint8_t sreg_save;

    // Save SREG, disable interrupts, do code, restore SREG.
    // If interrupts were enabled, they will be re-enabled.
    sreg_save = SREG;
    cli();                        // Turn OFF interrupts to avoid corruption
                                // during a multi-byte read.
    temp = totalMilliseconds;     // get a copy while interrupts are disabled.
    SREG = sreg_save;

    return temp;                  // return a 'clean' copy of the counter.
}

int main(void) {
    uint32_t ledTime = 0UL;

    /* set PA7 of PORT A as an OUTPUT pin. The other bits are left as '0'
     * so that their associated pins will be INPUT pins. */
    PORTA.DIR |= PIN7_bm;

    /* ** TCA Configuration as NORMAL counter with interrupt for 1ms OVF ** */
    TCA0_SINGLE_PER = 0x0D04;     // 3,333 - 1 since the counter starts at 0.
    /* enable overflow interrupt */
    TCA0.SINGLE.INTCTRL = TCA_SINGLE_OVF_bm;
    /* set Normal mode */
    TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_NORMAL_gc;
    TCA0.SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV1_gc /* set CLK_PER/64) */
                    | TCA_SINGLE_ENABLE_bm;    /* start timer */

    /* enable Global interrupts */
    sei();

    // Blink the LED at a rate set by the TCA overflow rate.
    while (1) {
        if( millis() > ledTime ) {
            ledTime = millis() + LED_DELAY;
            /* toggle the LED state. */
            PORTA_OUTTGL = PIN7_bm;
        }
    }
}

```

```
}
```

Compile the code and download it into the device. The LED is now being controlled by timing test in the while() loop using the millis() function based on the 1ms interrupt 'tick' from the TCA interrupt service routine.

As a simple test, change the value of LED_DELAY to change the LED flash rate.

This modified code is in the main05.c file. The complete project, ITMC_II_1234.X.zip, can be downloaded from "<https://www.gameactive.org/dist/>".

5. General Programming

With the basics of Input, Output, and Timing covered, the support code for these functions should be organized into their own files for better version and configuration control. This will be very helpful as the project grows and coding becomes more complex.

A common architecture may look like this:

- a. main.c - Overall project control.
- b. systime.c - time support and timer initialization functions.
 - systime.h - header file to be added to main.c to access time functions.
- c. io_ctrl.c - input/output control and initialization functions.
 - io_ctrl.h - header file to be added to main.c to access i/o functions.
- d. sysdefs.h - (optional) global definitions file.

Starting with main05.c having all of the source code, RMC on all of the other 'main' files and select "Remove from project" then rename main05.c to main.c. To create the two new files, use Source Files > New > avr-main.c > File Name: systime > Finish and Source Files > New > avr-main.c > File Name: io_ctrl > Finish

For the header files, use

Header Files > New > C Header File > File Name: systime > Finish and

Header Files > New > C Header File > File Name: io_ctrl > Finish and for the optional sysdefs.h file, use

Header Files > New > C Header File > File Name: sysdefs > Finish

The hierarchy tree should now show three Header Files and three Source Files.

Open main.c, systime.c, and systime.h open in the Editor by double clicking them if then are not already open.

In systime.c, delete all lines below the `#include <avr/io.h>` line.

Move, from main.c, the lines for

```
volatile static uint32_t totalMilliseconds;    // global variable
ISR(TCA0_OVF_vect)
{
    ...
}
uint32_t millis()
{
    ...
}
```

into the systime.c file. Also, add the code

```
#include "systime.h"
```

just under the `#include <avr/io.h>` line.

Add a function void `init_systime(void)` to the systime.c and move all of the TCA0 initialization code from main.c into it.

Put as prototype for it in systime.h and add `#include "systime.h"` to the main.c file.

Place a call to `init_systime()` where the TCA0 code was in main.c.

In the systime.h file, delete all lines of the `__cplusplus` `#if` conditional, unless you plan to code in C++ later.

Copy

```
#include <avr/interrupt.h>
```

from main.c and add it under the `#define SYSTIME_H` line.

Move the `uint32_t millis()` ; line from main.c to systime.h also.

For io_ctrl.c, remove the `main()` function code block. Add an `#include "io_ctrl.h"` statement.

Add a function called `void init_io(void){ }` and move the PORTA initialization code from main.c into it. Then add a call to `init_io()` where the code was.

In io_ctrl.h, remove the `cplusplus` code and add a prototype for `void init_io(void)`.

Compile the code and download it into the device. The LED is being controlled by timing test in the while() loop using the millis() function based on the 1ms interrupt 'tick' from the TCA interrupt service routine as before.

The current project ITMC_II_5.X.zip, reorganized in this method, can be downloaded from "<https://www.gameactive.org/dist/>". This is one of many way to organize your projects.

Other things that can be done:

- Move the _OUTTGL operation into a function called toggle_LED().

- Make led_ON() and led_OFF() functions.

- Make a blink_LED(int n) function to blink the LED n times at a 200ms rate.

SECTION III - Exploration of Unique Hardware

This section will look at the variety of hardware peripherals available in the AVR ATtiny family of MCUs and explore some of their features. These selections do not have to be reviewed in any particular order as they are essentially independent of each other.

1. Serial Communications - USART

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3216-Getting-Started-with-USART-DS90003216.pdf>

A common interface between a project and a PC or laptop is through a USB-to-Serial board. These are available from Adafruit and Sparkfun and come in 5v and 3.3v versions. Using the AAA battery pack as a power source, use a 3.3v board if using rechargeable AAAs, use a 5v board if using standard or alkaline AAAs. Alternately, the USB board can supply power to replace the battery pack.

Figure III-1 shows an example circuit used to explore the Serial Communications port (USART) of an ATtiny412. It uses the USB serial board for power instead of the battery pack.

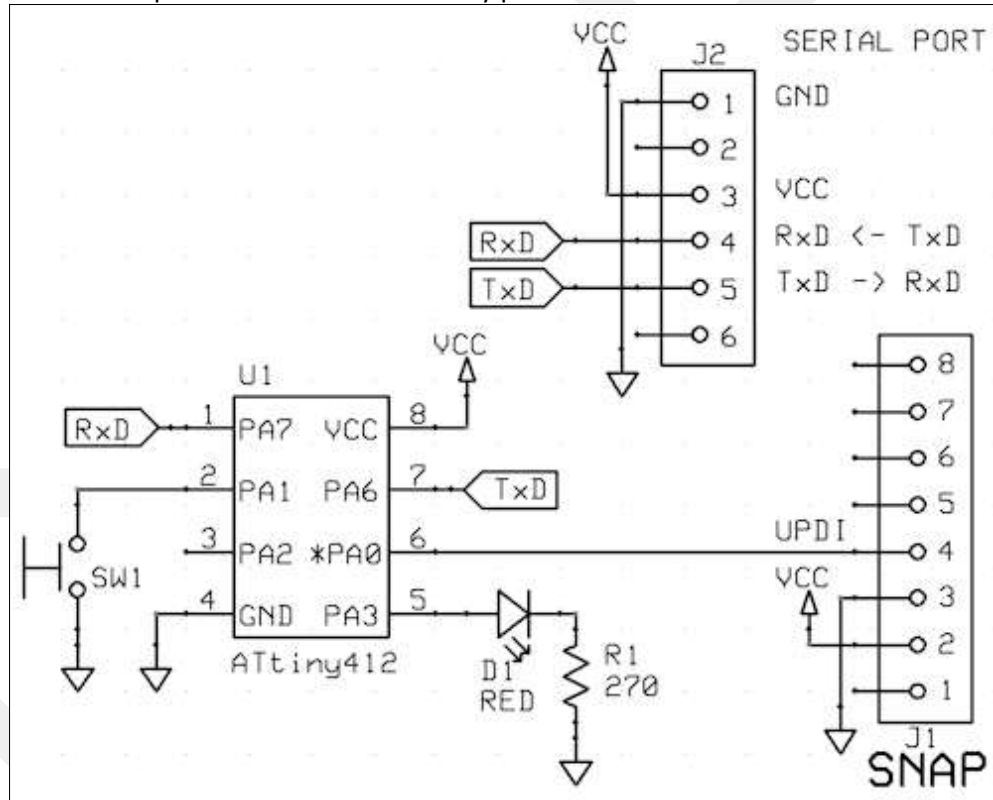


Figure III-1

The 6-pin serial port follows the FTDI standard and should be a male 0.1" header to match the female connector used on most USB-to-Serial boards.

The LED was moved to PA3 to free up PA7 since it is the receive (RxD) pin. PA6 is the transmit out (TxD) pin. (see Table 5.2 USART0)[2]

NOTE: Figure III-1 shows the SIOC BOARD pins numbers, NOT the ATtiny412 device pin numbers.

Connect the USB cable to the PC and in MPLAB X select Window > Debugging > Data Visualizer to bring up the MPLAB Data Visualizer window. A COM port should show up in the Serial Ports panel. Select the gear symbol to configure the port to 9600 8N1 if not already set that way. This tool can be used to display the serial output for this example.

Optionally, an Arduino board can be used as something to talk to. Connect RxD to Arduino TxD and TxD to Arduino RxD. Be sure to use the Arduino +5v power for the ATtiny412 instead of the battery pack.

Section 24[2] describes the USART operation and its registers. Two files are added to the project:

Header Files > SerialPoll.h

Source Files > serialPoll.c

These will contain the serial support code. A simple polling process will be used. See [ref] for ideas on how to implement an interrupt based system.

The USART will be set up for Full-Duplex, Asynchronous, 8N1 communications at 9600 baud. Section 24.3.1[2] lists the steps needed to initialize the USART for this mode.

a. Baud rate

USART0.BAUD sets the BAUD rate (i.e. the bit rate of communications). The following lines of code will calculate the baud rate based on the CPU clock rate.

```
#define F_CPU 3333333L // CLK_PER is 3.3 MHz
#define USART_BAUD_RATE(BAUD_RATE) ((float)(F_CPU * 64)/(16 * (float)BAUD_RATE)+0.5)
and
USART0_BAUD = (uint16_t)USART_BAUD_RATE(baud);
will set the baud rate.
```

b. Frame format

USART0.CTRLA sets the frame format. The register is set to Asynchronous 8N1 with predefined bit group code (_gc) found in the iotn412h file.

```
USART0_CTRLA = USART_CHSIZE_8BIT_gc;
```

c. TxD as an output pin

Set the DIR register to make pin PA6 an output. PA7 (RxD) will be an input by default.

```
PORTA_DIR |= PIN6_bm; // TxD as an OUTPUT
```

d. Enable transmitter and receiver

Use CTRLB to enable the receiver and transmitter.

```
USART0_CTRLB |= (USART_RXEN_bm | USART_TXEN_bm);
```

All of this can be gathered into an initialization subroutine as shown below.

```
/* Initialized USART0 registers and IO pins */
void USART0_init(uint16_t baud)
{
    /* Set IO port pin directions. */
    PORTA_DIR &= ~PIN7_bm; // Rx as INPUT (default)
    PORTA_DIR |= PIN6_bm; // Tx as OUTPUT

    /* Set baud rate */
    USART0_BAUD = (uint16_t)USART_BAUD_RATE(baud);

    /* Set frame format: 8bit Data, No Parity, 1stop bit (8N1) */
    USART0_CTRLA = USART_CHSIZE_8BIT_gc; // default

    /* Enable receiver and transmitter */
```

```

    USART0_CTRLB |= (USART_RXEN_bm | USART_TXEN_bm);
}

```

If a byte is written into the TXDATA register, the USART will transmit it out the Tx line. The register should be checked to see if it's empty before writing to it to prevent an overwrite. The example code below does this.

```

/* Blocking call to send one character. */
void USART0_sendChar(char data)
{
    /* Wait for empty transmit buffer */
    while ( !(USART0_STATUS & USART_DREIF_bm) );

    /* Put data into buffer, sends the data */
    USART0_TXDATA = data;
}

```

This is a 'blocking call' because the function does not immediately return and if the USART failed to send the last byte, it will wait forever. A way to mitigate this is to add a timeout to the wait operation.

To receive a byte, just check the USART_RXCIF bit in the STATUS register and read the RXDATA register if the bit is set. The example code below shows one way to do this.

```

/* Blocking call to receive one character.
 * USART0_isChar() should be called and return 'true' before calling this function.
 */
char USART0_recvChar()
{
    /* Wait for data to be received */
    while ( !(USART0_STATUS & USART_RXCIF_bm) );

    /* Get and return received data from buffer */
    return USART0_RXDATA;
}

// Check for character in Receive register
bool USART0_isChar()
{
    bool result = false;

    if( USART0_STATUS & USART_RXCIF_bm ) {
        result = true;
    }

    return( result );
}

```

All of this code is in the SerialPoll files included in the ITMC_III_1.X.zip project file. There is also a main.c file that uses these files to send out a stream on ASCII characters. The project file can be downloaded from "<https://www.gameactive.org/dist/>".

2. Analog-to-Digital Converter - ADC

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3209-Getting-Started-with-ADC-DS90003209.pdf>

NOTE: ADC input resistance is approximately 14k ohms and must be considered when attaching a sensor or other voltage generating circuit.

This family of devices provides a 10-bit Analog to Digital Converter (ADC) with a maximum sample rate of 115 kHz. The ADC generates a digital number relative to the voltage level applied. 0x000 for zero volts to 0x3FF for an input equal to the Vref voltage. A configuration is also provided to measure the internal temperature of the device. This example will measure the voltage from a variable resistor and use three LEDs to show its position. An adc.c. and an adc.h file will be added to the project to contain the ADC specific code.

The green LED will light if the voltage is less than $1/2$ Vref. The yellow LED will light if the voltage is between $1/2$ and $3/4$ Vref. The red LED will light if the voltage is greater than $3/4$ Vref.

Figure III-2a shows the circuit used in this example. The internal timer will trigger a voltage measurement once per second. The three colored LEDs will show the relative position of the wiper of the variable resistor. Figure III-2b shows a typical buffered ADC circuit using a single supply op-amp.

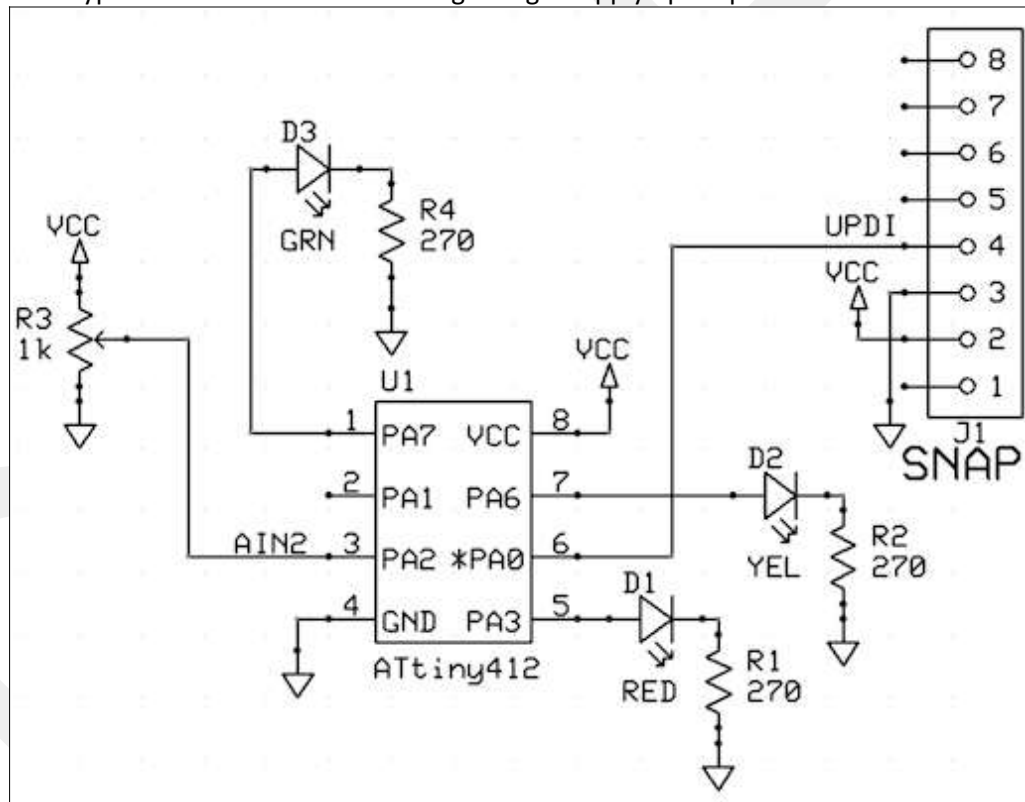


Figure III-2a

This circuit can be used to read the position of R3 and processor will determine which LED to light up based on the voltage measured.

a. Setting up the ADC

Section 30[2] describes the ADC. The ATtiny412 provides 6 channels for voltage measurement at a 10-bit resolution. Channel 2 (AIN2) is used in this example. Table 5-2[2] shows that AIN2 is connected to PA2. To simplify things, only 8-bits of the ADC will be used. The CTRLA.RESSEL will be set to '1' for this mode.

```
ADC0.CTRLA = ADC_RESSEL_8BIT_gc;
```

The ADC will be used in free-running mode. No accumulation will be used, so the CTRLB register is not set. This example will use 2.5V for Vref. This provides a wide range for both a Vdd of 3.6v or 4.5v. Set the VREF using the VREF.CTRLA register. The following code will do that.

```
VREF.CTRLA |= VREF_ADSCREFSEL_1_bm; // set Vref to 2.5v.
```

Section 30.3.2.2[2] states that the maximum sampling frequency is 1.5 MHz for 10-bit resolution. With CLK_PER at 3.3 MHz (default), a DIV4 will be used. The SAMPCAP will also be set to '1'. This code example will do these settings. (see 30.5.3)[2]

```
ADC0.CTRLA = ADC0.CTRLA | ADC_REFSEL_VDDREF_gc | ADC_PRESC_DIV4_gc;
```

Next, set the MUXPOS register to select AN2 (PA2 pin) as the input to the ADC.

```
ADC0.MUXPOS = ADC_MUXPOS_AIN2_gc;
```

Lastly, enable the ADC by setting the Enable bit in the CTRLA register.

```
ADC0.CTRLA |= ADC_ENABLE_bm; // enable the ADC.
```

The ADC is now configured to start measuring the input voltage when trigger by setting the STCONV bit in the COMMAND register.

```
ADC0.COMMAND = 0x01;
```

This bit will remain set while the ADC conversion process is in progress. Once the process sets it to '0', the data can be read from the Results register(s).

```
while (ADC0.COMMAND == 0x01); // wait for completion.  
voltage = ADC0.RESL;
```

This voltage is then tested to determine which LED to turn on. The following pseudo code shows one way to do this. Note: This is using 8-bit so maximum voltage is 0xFF.

If voltage < 0x80 then turn ON Green LED else turn it OFF.

If voltage >= 0x80 && voltage is < 0xC0 then turn ON YELLOW LED else turn it OFF.

If voltage >= 0xC0 then turn ON RED LED else turn it OFF.

Configure and control the two additional LEDs using the methods of Section II.3

To minimize the effects of the low input impedance of the ADC, a buffer can be used at the input as shown in

3. Digital-to-Analog Converter - DAC

ref: <https://ww1.microchip.com/downloads/en/DeviceDoc/TB3210-Getting-Started-with-DAC-90003210A.pdf>

The DAC generates a voltage proportional to the digital value in its DATA register. This example configures it to provide a voltage to an output pin. Its output can also be used to provide a reference voltage to the Analog Comparator(AC) and the Analog-to-Digital Converter(ADC).

The DAC can only source 1ma of current and will therefore not drive an LED. This example will require a VOM, DMM, or Oscilloscope to read and display the DAC output voltage. The millis() function will be used to generate a sawtooth waveform by incrementing a counter and letting it roll over to zero. The counter value will be sent to the DAC for output. Figure III-2a will be used with the LED removed so that PA6 can be used as the DAC output. A dac.c and a dac.h file are added to the project to contain the DAC specific code. Adjust the value of SAMPLE_DELAY to speed up or slow down the waveform frequency.

a. Setting up the DAC

Section 31[2] describes the DAC. The tinyAVR 1-series devices provide an 8-bit DAC that supports an update rate of 350 kHz. The DAC can be used to provide a reference voltage to the Analog Comparator(AC) and the Analog-to-Digital Converter(ADC). This example configures it to provide a voltage to an output pin.

First, select the Voltage Reference(VREF) to set the upper output level of the DAC with the DACOREFSEL bits in the VREF.CTRL0 register. The following code will do that.

```
// set both ADC and DAC Vref to 2.5v.  
VREF.CTRLA |= VREF_ADACOREFSEL_1_bm | VREF_DACOREFSEL_1_bm;
```

Now, enable the output pin and enable the DAC.

```
PORTA.DIR |= PIN6_bm; // set as OUTPUT  
DAC0.CTRLA |= DAC_OUTEN_bm | DAC_ENABLE_bm;
```

These two lines should be put into an init_dac() function in the dac.c file.

The last step is to provide a function to set the DAC output value. The following code is a way to do that.

```
void set_dac_output(uint8_t value)  
{  
    DAC0.DATA = value;  
}
```

An example of a simple count and output loop is shown below.

```
// Increment a counter and copy its value to the DAC to generate a voltage.  
uint8_t count = 0;  
  
while (1) {  
    if( millis() > ledTime ) {  
        ledTime = millis() + LED_DELAY;  
        toggle_LED();  
    }  
  
    if( millis() > sampleTime ) {  
        sampleTime = millis() + SAMPLE_DELAY;  
        /* update the count. */  
        ++count;  
        /* update DAC value */  
        set_dac_output(count);  
    }  
}
```

An example project using the code discussed is in the ITMC_III_3.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

DRAFT

4. Serial Peripheral Interface – SPI

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3215-Getting-Started-with-SPI-DS90003215.pdf>

NOTE: HOST = MASTER = **CONTROLLER** and CLIENT = SLAVE = **PERIPHERAL**

4a. SPI – Controller

This example will require an Arduino board or similar device to act as a SPI Client. The code will configure the ATtiny1614 as a SPI Host and send ASCII characters to the Arduino Client to be printed out in the Serial Monitor window. Section 25[6] has information about the SPI – Serial Peripheral Interface for both Host and Client configurations.

The SPI Host initiates a message transfer by asserting the /SS line for chip select. The Host then generates the required clock pulses on the SCK line while shifting data out through the MOSI line and/or receiving data through the MISO line. The Client SHOULD have its data ready to shift out before the /SS is asserted since the SCK pulses usually begin immediately after /SS is asserted. Figure 25-5[6] shows how the clock and data are related in each of the four possible SPI Data Transfer Modes. Mode 0 will be used in this example. The Host will also operate in NORMAL mode and will NOT be using interrupts.

Add a file spi_host.c to the Source File section of the project. New > newavr-main.c. Rename to spi_host.c > Finish. Delete the int main(void) code block and add

```
void init_spi(void){ }
```

and

```
bool spiSendByte(uint8_t val){ }
```

functions to the file. Then add an spi_host.h file, New > C Header File, to the Header Files section to store the function prototypes.

Identify the MOSI, MISO, SCK, and /SS pins for the device being used. For the ATtiny1614 14SI0C, these are:

Name	ID	Pin	Board Pin
MOSI	PA1	11	11
MISO	PA2	12	12
SCK	PA3	13	13
/SS	PA4	2	1

Configure the MOSI, SCK, and /SS pins as OUTPUTs. Place this code init_spi().

```
PORTA.DIR |= PIN1_bm | PIN3_bm | PIN4_bm;
```

In init_spi(), configure the device as a Master, set the pre-scaler to divide CLK_PER by 64 to get a 52 kHz SCK clock. A faster clock can be used if the circuitry is designed to handle higher speeds. The following code is an example of the setup:

```
SPI0.CTRLA |= SPI_MASTER_bm | SPI_PRESC_1_bm;
```

Turn OFF Buffer operations, disable mode switch on /SS, and use Mode 0.

```
SPI0.CTRLB |= SPI_SSD_bm; // Default Mode 0.
```

The SPI can now be enabled with:

```
SPI0.CTRLA |= SPI_ENABLE_bm;
```

Then initiate a dummy transfer to set the IF flag in the INTFLAGS register. This can then be tested for a clear DATA register before trying to send more data.

```
SPI0.DATA = 0; // initiate first transfer. No /SS.
```

Per 25.3.2.1.1[6], SPI0.INTFLAGS IF bit is set after transfer has completed. Test and reset this bit before attempting to send more data.

Write data into the SPI0.DATA register. This will initiate data transfer IF the ENABLE bit in the SPI0.CTRLA register is set to '1'. This logic will be needed to send a byte. The following code, placed in sendByte(), is an example of how this can be done:

```
bool result = false;

PORTA.OUT &= ~(_SS); // activate /SS
SPI0.DATA = val;
while((SPI0.INTFLAGS & SPI_IF_bm) == 0); // wait for transfer to complete
PORTA.OUT |= _SS; // deactivate /SS

// Check for collision.
if(SPI0.INTFLAGS & SPI_WRCOL_bm) {
    result = false;
} else {
    result = true;
}

return result;
```

Set up a simple loop in main() to send an ASCII character to the Slave once a second and toggle the LED on PA7 each time a value is sent. Example code is given below.

```
uint32_t ledTime = 0UL;
uint8_t val = 'A';

init_sys_time(); // set up TCA0 timer.
init_io(); // set up IO pins.
init_spi(); // set up SPI as a Host.

/* enable Global interrupts */
sei();

// Send out voltage, in HEX, each second.
while (1) {
    if( millis() > ledTime ) {
        ledTime = millis() + LED_DELAY;

        spiSendByte(val);
        if( ++val > 'Z' ) {
            val = 'A';
        }
        toggle_LED();
    }
}
```

Configure the Arduino to act as a SPI Client to receive the data. This has to be done manually as the Arduino SPI does not support Client operations. (see Arduino_SPI_Client.zip). Figure 4a shows the system connections.

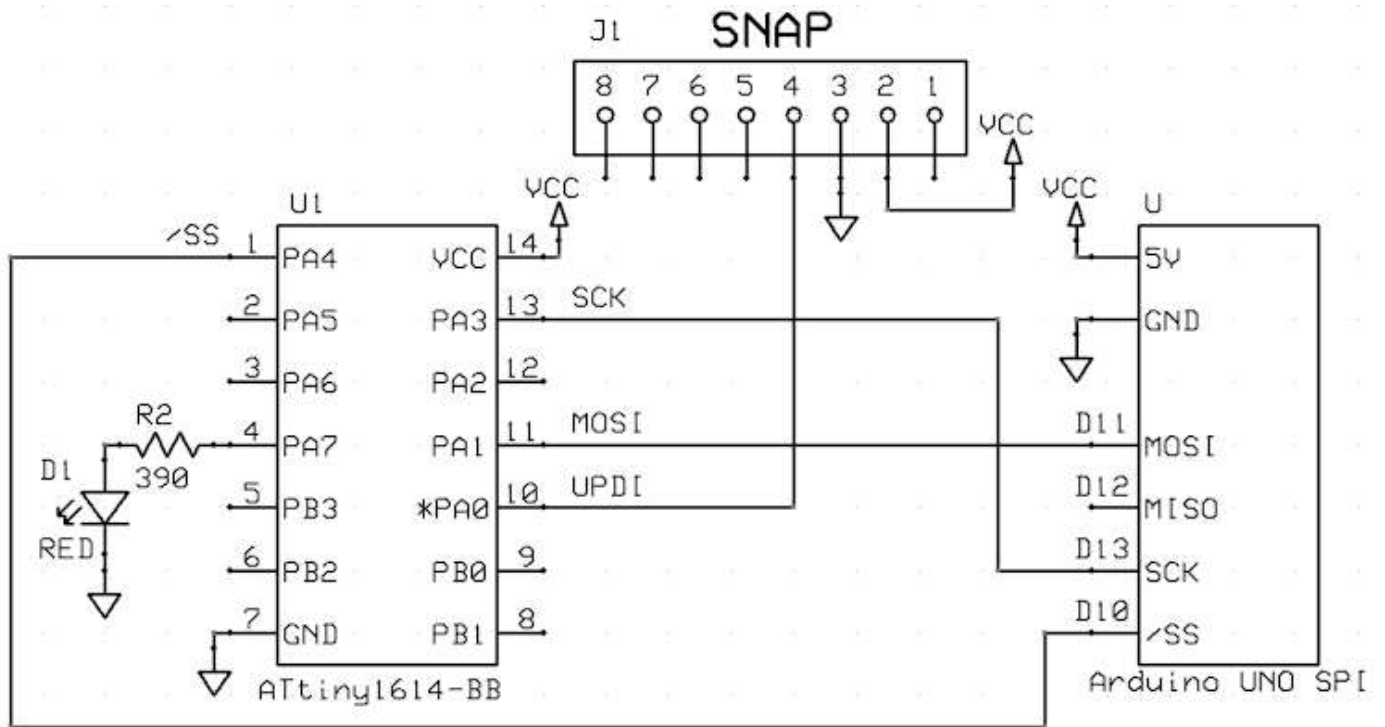


Figure III-4a

The Arduino is set up as a SPI Client will read its SPI Data In register and print the character received to the Serial Monitor window. An example of this in the Arduino_SPI_Client.zip folder, along with the code from this section in project ITMC_III_4a.X.zip, can be downloaded from " <https://www.gameactive.org/dist/> ".

4b. SPI - Peripheral

This example will require an Arduino board or similar device to act as a SPI Host device. The code will configure the ATtiny1614 14SIOC as a SPI Client. It uses PA4 instead of PA0 (RESET/UPDI) for the /SS line and reads ASCII characters from the Arduino to control the LED on PA7.

NOTE: HOST = MASTER = **CONTROLLER** and CLIENT = SLAVE = **PERIPHERAL**

The SPI Client waits for the /SS line to be asserted chip select. The Host then generates the required clock pulses on the SCK line while shifting data out through the MOSI that is connected to the Client MISO line. If the Client is sending data back to the Host, it MUST have its data ready to shift out before the /SS is asserted since the SCK pulses usually begin immediately after /SS is asserted. Figure 25-5[6] shows how the clock and data are related in each of the four possible SPI Data Transfer Modes. Mode 0 will be used in this example. The Client will use interrupts to automatically read and store the data. A flag will be set when there is data available from the Host.

Add a file spi_host.c to the Source File section of the project. New > newavr-main.c. Rename to spi_host.c > Finish. Delete the int main(void) code block and add

```
void init_spi(void){ }
```

and

```
bool spiSendByte(uint8_t val){ }
```

functions to the file. Then add a spi_host.h file, New > C Header File, to the Header Files section to store the function prototypes.

Identify the MOSI, MISO, SCK, and /SS pins for the device being used. For the ATtiny1614 14SIOC, these are:

Name	ID	Pin	Board Pin
MOSI	PA1	11	11
MISO	PA2	12	12
SCK	PA3	13	13
/SS	PA4	2	1

Configure the MISO as an OUTPUTs if data is to sent back to the Host. Place this code init_spi().

```
PORTA.DIR |= PIN2_bm;
```

In init_spi(), configure the device as a Client and enable it. The following code is an example of the setup:

```
SPI0.CTRLA |= SPI_ENABLE_bm;
```

Clear the data received flag and enable the SPI interrupt.

```
SPI0.INTFLAGS = SPI_IF_bm;           // clear IF flag
SPI0.INTCTRL |= SPI_IE_bm;          // enable interrupt
```

Add an interrupt handler to read incoming data and set a data ready flag.

```
ISR(SPI0_INT_vect)
{
    spiDataIn = SPI0.DATA;
    spiInFlag = true;
    SPI0.INTFLAGS = SPI_IF_bm;        // clear IF flag
}
```

Add a function to allow accessing the data ready flag and one for returning the last data received. Examples are give below:

```
bool isSpiData()
```

```

{
    return spiInFlag;
}

// Check isSpiData() first. If true, then call this function.
uint8_t getSpiData()
{
    spiInFlag = false;           // reset flag.
    return spiDataIn;
}

```

Write a simple loop in main() to check for data ready and read the data. Have the data control the LED. An example is given below:

```

if (isSpiData()) {
    val = getSpiData();
    if (val == 'A') {
        set_LED(true);
    }
    if (val == 'B') {
        set_LED(false);
    }
}

```

Configure the Arduino to act as a Host to send alternating 'A's and 'B's. (see Arduino Help > Reference > SPI). Figure 4b shows the system connections.

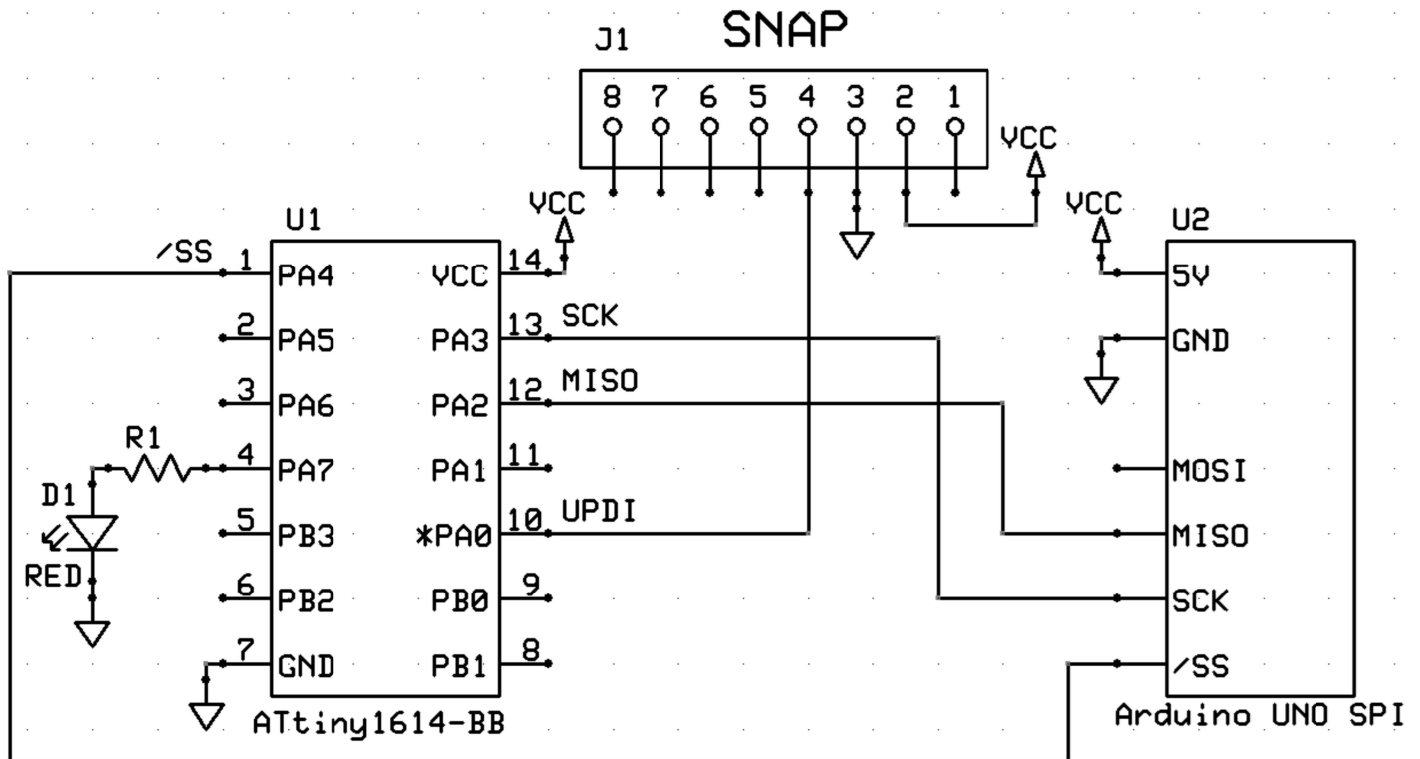


Figure III-4b

The Arduino is set up as a SPI Host to send data through its SPI Data Out register to the Client. It could also print the character sent in the Serial Monitor window. An example of this in the Arduino_SPI_Host.zip folder, along

with the code from this section in project ITMC_III_4b.X.zip, can be downloaded from "<https://www.gameactive.org/dist/>".

DRAFT

5. Two-Wire Interface - TWI (I2C)

ref: https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ApplicationNotes/ApplicationNotes/atmel-2565-using-the-twi-module-as-i2c-slave_applicationnote_avr311.pdf

This example will require an Arduino board or similar device to act as a Master I2C controller. The code will configure the ATtiny1614 as an I2C Slave at address 0x50 and allow control of the LED and reading the switch state. Figure 1 shows the connections with the Arduino supplying power.

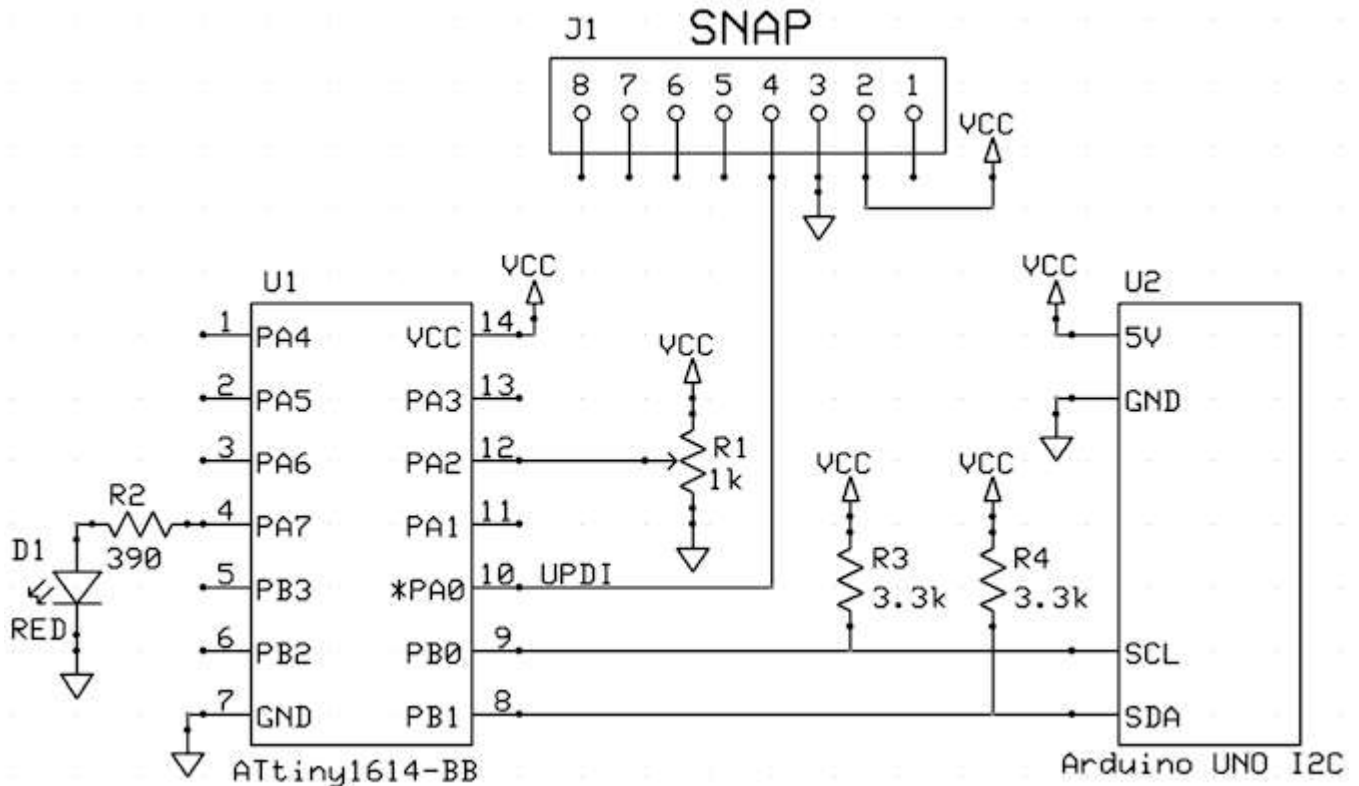


Figure III-5

This example uses code for a register based I2C Slave. The source code is part of the ITMC_III_5.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

This code makes the device look like an I2C slave with 16 data registers. The I2C message format is to send the start register number with a Write (SDA_W) command followed by 'n' number of Read (SDA_R) commands where 'n' is the number of registers to read starting with the start register number. Register 0 is always set to 0. The other 15 registers can contain data. In this example, register 1 contains the upper 8-bits of the ADC value based on the position of the R1 wiper.

twiRegSlave uses a state machine architecture to service the TWI0_SSTATUS register following a TWI0_TWIS interrupt. The status register indicate the cause of the interrupt and follows the state transitions of various I2c messages allowing the system to extract data from them.

To Read a register, the I2C Master uses the following message format:

SDA_W REGNUM SDA_R DATA [DATA] ...

Example: Read Register 1

0x50_W 0x01 0x50_R 0xNN

Example: Read Registers 4-6

```
0x50_W 0x04 0x50_R 0xNN 0xQQ 0xRR
```

To Write to a register, the I2C Master uses the following message format:

```
SDA_W REGNUM DATA [DATA] ...
```

Example: Write to Register 1

```
0x50_W 0x01 0xNN
```

Example: Write Registers 3-4

```
0x50_W 0x03 0xNN 0xQQ
```

The main() is a simple sample loop that updates the contents of register 1 based on TWI_DELAY. It also provides a 'heartbeat' using the LED at a rate set by LED_DELAY. After initialization, this is the loop code. The WDT is used to recover from I2C start-up synchronization mis-match.

```
// Read voltage. Set register 1 based on value.
while (1) {
    resetWDT();

    // Heartbeat
    if( millis() > ledTime ) {
        ledTime = millis() + LED_DELAY;
        toggle_LED();
    }

    if( millis() > ledTime ) {
        twiTime = millis() + TWI_DELAY;

        /* sample the input voltage. */
        trigger_adc();
        /* wait for conversion to complete and return voltage. */
        voltage = read_adc();

        // update data register 1.
        twiSetRegister(1, voltage);
    }
}
```

The Arduino is set up as an I2C Master. It will periodically send a read register 1 message to the Slave at 0x50 and print the return data in the Serial Monitor window. An example of this is in the Arduino_I2C_Master folder that can be downloaded from " <https://www.gameactive.org/dist/> ".

6. Other Timer/Counters

a. TCB

ref: <https://ww1.microchip.com/downloads/en/DeviceDoc/TB3214-Getting-Started-with-TCB-90003214A.pdf>

The TCB counter is designed to measure frequency and pulse width of external waveforms. It also has PWM capabilities and can be used to monitor processing time of internal functions.

In this example, the TCB timer will be used to support the `millis()` function. This might be needed if the TCA timer was being used to generate multiple RC servo signals as it can easily generate six of them. Section 21[2] describe the many other modes of operation for this timer.

The set up is very similar to configuring TCA in Section II-4. Figure I-1 hardware will use the ATtiny412. The TCB will now be used to increment the `totalMilliseconds` variable. The TCB configuration code can replace the TCA code in the `sys_time.c` and `sys_time.h` files or can be `#if .. #else .. #endif` for conditional compiling.

```
/* *** TCB Configuration as NORMAL counter with interrupt for 1ms OVF *** */
void init_sys_time(void)
{
    // The CCMP register contains the TOP value of the counter.
    TCB0.CCMP = 0x0D04;          // 3,333 - 1 since the counter starts at 0.
    /* enable periodic interrupt interrupt */
    TCB0.INTCTRL = TCB_CAPT_bm;
    /* set mode */
    // TCB0.CTRLB = (default, mode is periodic interrupt.)
    // TCB0.CTRLA = (default, use CLK_PER 3.333 MHz)
    TCB0.CTRLA |= TCB_ENABLE_bm; // turn ON the counter.
}

/* TCB interrupt service routine. */
ISR(TCB0_INT_vect)
{
    ++totalMilliseconds;
    TCB0.INTFLAGS = TCB_CAPT_bm; // clear the interrupt flag
}
```

An example project using the code discussed is in the ITMC_III_6a.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

b. TCD

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3212-Getting-Started-with-TCD-DS90003212.pdf>

The TCD timer is a 12-bit counter system designed for use in switching power supplies and motor controllers when paired with dual MOSFETs or with half-bridge or full-bridge driver circuits.

This example will use the Two Ramp Mode described in section 22.3.3.2.2 of the data sheet[2]. The two outputs, WOA (PA6) and WOB (PA7), will be used to independently control the brightness of two LEDs. This method could be used to control the power levels of two independent MOSFETs in a switch power supply or motor drive circuit.

The TCD sequences through four states to complete one counter cycle: Dead time WOA (WOA=LOW), On time WOA (WOA=HIGH), Dead time WOB (WOB=LOW), and On time WOB (WOB=HIGH). The various Modes control when these state occur. This example will use the One Ramp Mode with the Counter and Delay prescalers set the same.

First, set the IO pin for WOA and WOB as OUTPUTs.

```
PORTA.DIRSET = PIN6_bm | PIN7_bm;
```

The clock source will be CLK_PER and is selected by the CTRLA register CLKSEL[1:0] bits. The following code sets up the CTRLA register.

```
TCD0.CTRLA = TCD_SYSCLK_gc | TCD_CNTDIV4_gc | TCD_SYNCDIV4_gc;
```

The CTRLB register controls the Waveform Generation Mode. The following code configures the counter for One Ramp mode.

```
TCD0.CTRLB = TCD_ONERAMP_gc;
```

In One Ramp Mode, the period for WOA is set by the CMPACLR register and the period for WOB is set by the CMPBCLR register. The on-times are set by the CMPASET and CMPBSET registers respectively. (see Figure 22-3[2]) The following code will set up the counter to generate two 25% duty cycle waves.

```
TCD0.CMPASET = 0x0400;  
TCD0.CMPACLR = 0x0800;  
TCD0.CMPBSET = 0x0C00;  
TCD0.CMPBCLR = 0x0FFF;
```

Finally, turn on the timer by setting the ENABLE bit to '1'.

```
TCD0.CTRLA |= TCD_ENABLE_bm;
```

Figure III-2a can be used to show the two LEDs changing in brightness as the values of CMPASET and CMPBSET are changed in the main loop by calling setTCD_setA() and setTCD_setB() respectively.

```
void setTCD_setA(uint16_t val)  
{  
    TCD0.CMPASET = val;  
}  
  
void setTCD_setB(uint16_t val)  
{  
    TCD0.CMPBSET = val;  
}
```

Add code to the main loop to periodically change the set times using the `millis()` function as a timer. Do not use `toggle_LED()` as the TCD is now controlling the PA7 LED line.

An example project using the code discussed is in the ITMC_III_6b.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

7. Watchdog Timer - WDT (and Sleep Controller - SLPCTRL)

The Watchdog Timer (WDT)(Section 19[2]) is very useful in recovering from a 'code lockup'. If it is not triggered within its timeout period, it will reset the system. So, it is typically triggered within the main loop. The WDT can also be used for battery management. The WDT can reset a system that has been put in SLEEP mode where just about everything has been shut down to reduce power usage. The WDT is clocked by an independent ultra-low power oscillator. (section 19[2])

The first example will use the Sleep Controller (SLPCTRL)(Section 11[2]) and WDT to blink an LED every 8 seconds. The number of blinks will be equal to the number of 8 seconds intervals that the system has been running. The number will be reset back to 1 after 10 cycles. The circuit in Figure I-1 will be used.

The second example will use a switch to wake up the device from a SLEEP mode using a pin change interrupt. After waking up, the device will flash the LED and increment a counter maintained in SRAM.

a. WDT setup

The WDT is disabled by default at reset. Its configuration is controlled by the FUSE Configuration bits. In this example, the WDT will be set to NORMAL mode with an 8s timeout. Section 6.10.4.1[2] shows the configuration register WDTCFG. "These bit are copied into WDT.CTRLA at Reset". Therefore, the Configuration Bits are used to set up the WDT. Setting the PERIOD[3:0] bits to 0xB will put the WDT into NORMAL mode with an 8s timeout. The following operations are used to do this:

Production > Set Configuration Bits > (Right Side Bar) Read Configuration Bits > Option:PERIOD = 8KCLK. This will set

PERIOD Value = B

> Generate Source Code to Output.

Copy and paste the FUSES and LOCKBITS code into main.c, just under the #include <avr/io.h> statement. This will set the FUSES of any new device you program.

The counter is maintained in EEPROM. It is recovered on a WDT reset. It is reset to 0 on a Power-On reset. On use, it is incremented, tested, and updated back to EEPROM.

This is an example of setting the SLEEP mode to Power-down mode.

```
// Initialize Sleep Control for Power-Down mode.          PDOWN | SEN
SLPCTRL.CTRLA = SLPCTRL_SMODE_1_bm | SLPCTRL_SEN_bm;
```

The main() function checks for the type of reset that occurred and updates the count as needed. It then initializes the I/O for the LED and sets up the SLEEP mode. Then it services the counter and blinks the LED. The last step is to put the device into SLEEP mode and wait for the WDT to reset the system.

One possible use for this type of code might be to make very long timer that triggers a camera once every 24 hours to take a picture. Just divide the number of seconds in a day by 8.2 seconds of the WDT. Since the device is asleep most of the time, it would greatly improve the battery life.

There is also a <avr/wdt.h> file that has a wdt_reset() function use to reset the WDT when using the WDT as a 'code -lockup' detector. Just call it in your main loop.

An example project using the code discussed is in the ITMC_III_7a_WDT.X.zip project file that can be downloaded from " <https://www.gameactive.org/dist/> ".

b. SLPCTRL setup

"The contents of the register file, SRAM, and registers, are kept during sleep."(section11.2[2]). This is very important as it allows us to store data during Sleep and update that data after waking up.

After the initialization code process, the system is put to sleep with an assembly code directive 'asm("SLEEP");'. It will wake up on a Pin Change Interrupt caused by the switch on PA1. The LEVEL mode must be used. The PORTA pin has to be configured (see 16.5.11[2]) to cause an interrupt while SLEEP mode is active. An example of all of this is shown below.

NOTE: Don't forget to disable the WDT with FUSE settings and enable global interrupts.

```
int main(void) {
    uint8_t cycleCounter = 0;

    init_io();                // set up IO pins.

    sei();

    // Initialize Sleep Control for Power-Down mode.          PDOWN | SEN
    SLPCTRL.CTRLA = SLPCTRL_SMODE_1_bm | SLPCTRL_SEN_bm;

    while(1) {
        if( ++cycleCounter > 10) {
            cycleCounter = 1;
        }
        for (int k = 0; k < cycleCounter; k++) {
            flash_LED();
        }
        asm("SLEEP");        // wait for key press to wake up.
    }
}
```

In io_ctrl.c,h add:

```
in init_IO()
{
    ...
    /* enable the internal Pull-up resistor for PORTA pin 1 */
    // allow pin to wake up device while in SLEEP mode.
    PORTA.PIN1CTRL |= PORT_PULLUPEN_bm | PORT_ISC_LEVEL_gc;
    PORTA.INTFLAGS |= BUTTON_PIN_BM;        // clear flag
}

void enablePA1(void)
{
    PORTA.PIN1CTRL = PORT_PULLUPEN_bm | PORT_ISC_LEVEL_gc;
}

void disablePA1(void)
{
    PORTA.PIN1CTRL = PORT_PULLUPEN_bm;
}

ISR(PORTA_PORT_vect)
{

```

```

    disablePA1();
    // only PA1 used, so no tests needed. The interrupt wakes up the device.
    PORTA.INTFLAGS |= BUTTON_PIN_BM;          // clear flag
}

// Return true if BUTTON_PIN_BM is LOW.
bool isSwitchPressed()
{
    bool result = false;
    if ( (PORTA.IN & BUTTON_PIN_BM) == 0 ) {
        result = true;
    }
    return(result);
}

```

This type of code could be used in a remote event counter. SLEEP mode could extend battery the life. Because of the debounce problem, I would suggest a 'clean' digital pulse to trigger PA1.

An example project using the code discussed is in the ITMC_III_7b_SLPCTRL.X.zip project file that can be downloaded from " <https://www.gameactive.org/dist/> ".

8. Real Time Counter - RTC

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3213-Getting-Started-with-RTC-DS90003213.pdf>

The RTC can be used to keep track of real time since it runs even during Low-Power Sleep mode. Using the Internal Ultra Low-Power 32.768 kHz oscillator run through a divide by 32 counter with the 15-bit prescaler set for divide by 512, the RTC can generate an interrupt each half second and toggle the LED. Figure I-1 will be used in the example.

The RTC.CLKSEL Clock Select bits (CLKSEL) control the clock source. The default is INT32K, the internal 32 kHz OSCULP32K. To guarantee its startup, set the RUNSTDBY bit in the CLKCTRL.OSC32KCTRLA register.

```
CPU_CCP = CCP_IOREG_gc; // unlock Change Protected IO Registers (0xD8)
CLKCTRL.OSC32KCTRLA = 0x02; // Force startup of oscillator.
```

Then set the clock source in RTC.CLKSEL to use the internal 32kHz oscillator.

```
RTC.CLKSEL = RTC_CLKSEL_INT32K_gc; // 32.768kHz Internal (0: INT32K)
```

Once this is done, the code needs to wait for the system to 'sync up' the clock system. Monitor the RTC.STATUS register until this happens.

```
while( RTC.STATUS > 0); // Wait for RTC to synchronize with system.
```

The RTC.PER register is then set to 512 to detect the overflow condition. Every half-second.

```
RTC.PER = 512;
```

Next, the overflow interrupt bit is set to enable an interrupt on overflow,

```
RTC.INTCTRL |= RTC_OVF_bm;
```

and an interrupt service routine is added to toggle the LED each half-second.

```
ISR(RTC_CNT_vect)
{
    RTC.INTFLAGS = RTC_OVF_bm;
    toggle_LED();
}
```

RTC.CTRLA is used to set the prescaler to DIV32 to reduce the clock to 1024 Hz and enable the RTC. using the following code:

```
RTC.CTRLA = RTC_PRESCALER_2_bm | RTC_PRESCALER_0_bm // 0x05 for DIV32
            | RTC_RTCEN_bm; // Enable the RTC
```

To organize the code, add rtc.c to the Source Files area of the project and rtc.h to the Header Files area. Place all of the initialization code into a function called init_rtc() and the prototypes in the rtc.h file. If the avr/io.h does not have the defines for _gc labels, create them with _bm defines and place them in the rtc.h file.

Another use of the RTC is as an approximate 1ms millis() timer to free up the TCA for other duties if the system does not need an exact 1ms tic.

An example project using the code discussed is in the ITMC_III_8_RTC.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

9. Configurable Custom Logic - CCL

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3218-Getting-Started-with-CCL-DS90003218.pdf>

The CCL is a Look-Up Table (LUT) based logic block with two LUTs. These can be treated as customizable three input logic gates. There is also a configurable latch on the output. The ATtiny412 8SIOC can be used for this example. Section 28 of the data sheet [2] describes the CCL features.

The first two examples are for using the CCL as a logic gate. The selection is though the mode parameter in `init_CCL(char mode)`.

The LUT0 TRUTH table contains eight possible selections indexed by the three inputs. It outputs a '1' when an enabled selection is indexed. The bit pattern written to the table is used to enable selections. The output is sent to LUT0-OUT.

This example will use LUT0. LUT0 TRUTH0, 28.5.6[5], will be configured as a two input AND gate and use two switches as inputs. The input pins will use the INVERT mode to generate a HIGH when the switch is closed. The output of LUT0 will control an LED used as the output indicator. When both switches are closed, the AND gate output goes HIGH and LED stays ON.

LED	PA6	LUT0-OUT
SW1	PA1	LUT0-IN1
SW2	PA2	LUT0-IN2

NOTE: LUT0-OUT may be on another pin for other devices.

Make `ccl.c` and `ccl.h` files for the code and prototypes. This example code sets up the CCL an AND gate and configures LUT0 to use PA1 and PA2 as inputs.

```
CCL.LUT0CTRLA &= ~CCL_ENABLE_bm;      // disable LUT0
CCL.LUT1CTRLA &= ~CCL_ENABLE_bm;      // disable LUT1
// PA6 as an OUTPUT
PORTA.DIR |= PIN6_bm;
// Enable PullUps and Invert
PORTA.PIN1CTRL = PORT_PULLUPEN_bm | PORT_INVEN_bm; // INV, Enable PullUp
PORTA.PIN2CTRL = PORT_PULLUPEN_bm | PORT_INVEN_bm; // INV, Enable PullUp
// configure PA1 and PA2 as LUT0 inputs.
CCL.LUT0CTRLB = CCL_INSEL1_2_bm | CCL_INSEL1_0_bm; // Use IO (0x5) PA1
CCL.LUT0CTRLC = CCL_INSEL2_2_bm | CCL_INSEL2_0_bm; // Use IO (0x5) PA2
// Set up Truth Table. Output 1 if PA2 AND PA1 are 1.
CCL.TRUTH0 = 0b01000000; // 110 = 1, 0 for all other selections
/* or
// Set up Truth Table. Output 1 if PA2 OR PA1 OR BOTH are 1.
// CCL.TRUTH0 = 0b01010100; // 100=1, 010=1, 110=1, 0 for all other
*/
// Enable output pin
CCL.LUT0CTRLA |= CCL_OUTEN_bm;

CCL.LUT0CTRLA |= CCL_ENABLE_bm; // enable LUT0
CCL.CTRLA |= CCL_ENABLE_bm;    // enable CCL }
```

A suggestion is to use the LINK option for inputs that connects LUT1 output to LUT0 input. If TCB is used as an input INSEL0 of LUT1, it could modulate the LUT0 output. LINK LUT1 output to LUT0 as a third input. LUT1 can use TCB0 WO1 as Input. `CCL.LUT0CTRLB |= 0x07 INSEL0[0:3]`. Set `TCB.COMP` to `0xFFFF` and `DIV64` for a fast blink.

An example project using the code discussed and including more modes, is in the ITMC_III_9_CCL.X.zip project file that can be downloaded from " <https://www.gameactive.org/dist/> ".

10. Analog Comparator - AC

ref: <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ApplicationNotes/ApplicationNotes/TB3211-Getting-Started-with-AC-DS90003211.pdf>

In this example, a POT and two LEDs are used to show threshold detection using the Analog Comparator (AC) module (see Section 29[2]). The internal Vref will be selected for the negative input to the AC and is set to 2.5v. A simple polling loop will be used instead of interrupts. When the input voltage from the POT is less than 2.5v, the Green LED is ON and the Red LED is OFF. When the input voltage is greater or equal 2.5v, the Green LED is OFF and the Red LED is ON.

The Yellow LED is used as a 'heartbeat' to show that the program is running.

Pin and port connections as shown in Figure III-10. Use PA7 AINP0 for AC input. Internal Vref for compare.

Port Pin	Board	Description
PA1	2	RED
PA3	5	GRN
PA6	7	YEL
PA7	1	AINP0 (see Table 5-2 [2])

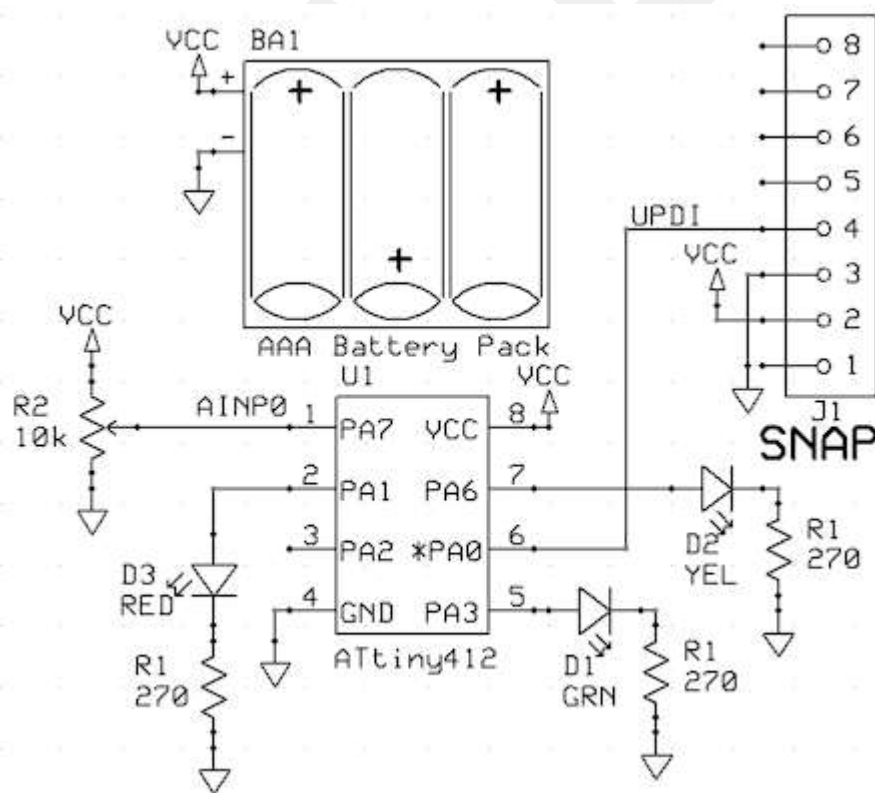


Figure III-10

First, use VREF0.CTRLA to set the Vref to 2.5v.

```
VREF.CTRLA = 0x02; // 2.5v ref (see Section 18.5.1[2])
```

Then configure the AC to use the Vref voltage on the negative input.

```
AC0.MUXCTRLA = 0x02; // VREF on negative input.
```

Lastly, enable the ac module using the AC0.CTRLA register.

```
AC0.CTRLA = AC_ENABLE_bm;
```

The output state of the AC can be checked by reading AC0.STATUS and testing bit 4.

```
State = AC0.STATUS & 0x10;
```

Don't forget to modify io_ctrl.c to use PA6 for the toggle LED and add code to control the RED and GREEN LEDs.

The example code below show one way to do this in main() to control the LEDs.

```
/* test the input voltage. */  
if( check_ac() ) {  
    setRedState(true);  
    setGreenState(false);  
} else {  
    setRedState(false);  
    setGreenState(true);  
}
```

An example project using the code discussed is in the ITMC_III_10_AC.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

11. Brown Out Detector - BOD

The BOD consist of two comparators. Each is monitoring the Vdd voltage. The BOD section compares Vdd to a fixed FUSE settable BOD threshold reference while the VLM uses an adjustable value that is set to a percentage above the BOD threshold. The BOD can be set to continuously monitor or to periodically sample the Vdd voltage.

Passing the BOD threshold will always cause a system reset. The VLM interrupt can be set to trigger on a falling, rising, or any crossing of the VLM threshold. Section 17[2] describes the capabilities of BOD module.

This example will use the VLM interrupt to trigger an SOS flashing signal by the LED if Vdd goes below the VLM threshold and turn off the flashing if Vdd then rises above the VLM threshold. System shutdown code would be used in place of the SOS code.

Use a 1K pot to vary Vdd from the battery to the AVR ATtiny device.

In battery powered systems, a BOD can be useful to gracefully shut down the system if the VCC voltage drops too low for reliable operation.

12. Accessing EEPROM

The Nonvolatile Memory Controller (NVMCTRL) is used to access the EEPROM as described in section 9.[2] All AVR ATtiny series-1 devices have at least 64 bytes of EEPROM. This is nonvolatile memory that retains its data even if power is lost.

NOTE: When the EEPROM is 'erased', all memory bits will be '1'.

Section 6.2[2] shows the EEPROM memory to start at address 0x1400.

To write data to EEPROM, write the data into the buffer. Check the EEBUSY flag, then activate the Configuration Change Protection (CPU.CCP) key (SPM:0x9D) and set the WP command (0x01) in the NVMCTRL.CTRLA CMD[2:0] section.

The `avr/eeprom.h` library does all of this in its

```
eeprom_write_byte((* uint8_t) address, data)
```

and takes care of adjusting the memory address so the starting address is 0x0000. To read EEPROM, use

```
eeprom_read_byte((* uint8_t) address)
```

See the `avr/eeprom.h` file for other functions.

Using Figure III-1 for this example, the following code snippet will write thirteen characters into EEPROM and then read them all back at once and output them to the Serial port. The Data Visualizer can be used to monitor the action.

```
// Write 13 character into EEPROM, then read them all back and output to Serial.
while (1) {
    if( millis() > ledTime ) {
        ledTime = millis() + LED_DELAY;
        eeprom_write_byte((uint8_t *)count, val); // avr/eeprom takes care of setting
                                                    // CCP and Write address register.

        ++count;
        if( ++val > 'Z' ) {
            val = 'A';
        }
    }
    if(count > 12) {
        for (int i=0; i<13; i++) {
            epval = eeprom_read_byte((uint8_t *)i); // avr/eeprom lib ASSUMES 0x1400
                                                    // as a base address.

            /* Send an ASCII character out. */
            USART0_sendChar(epval);
        }
        USART0_sendChar(0x0D);
        USART0_sendChar(0x0A);
        count = 0;
    }
}
```

An example project using the code discussed is in the ITMC_III_12.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

13. Accessing Flash Memory

The Nonvolatile Memory Controller (NVMCTRL) is used to access the FLASH as described in section 9.[2] To use 'C' to access FLASH, include the `prmspace` header and use the `pgm_read_byte_near()` and `pgm_read_word_near()` functions. Define data storage using `const`.

NOTE: The 'adrs' is an unsigned 16-bit value. Cast the location using `(uint16_t)`.

```
#include <avr/pgmspace.h>

const uint8_t data[] = {1, 2, 3, 4};

uint8_t val;
uint16_t adrs;

adrs = (uint16_t)data;
is the same as
adrs = (uint16_t)&data[0];

pgm_read_byte_near( adrs );    // returns an 8-bit byte.
pgm_read_word_near( adrs );    // returns a 16-bit word.
```

See Section IV-1 for example code.

SECTION IV - Example Projects

1. 20 LEDs controlled with 5 I/O lines.

This project uses 5 I/O line to control 20 LEDs using a differential drive technique and multiplexing. Each of the LEDs are kept ON for 1ms with a 20ms refresh rate (50 Hz). A lookup table is used to store the I/O configuration for each LED to tell the system which pair of lines to use for a given LED. It is important to disable the other three I/O lines when enabling the pair for the LED or other LEDs will light up.

The select[] table has twenty entries, one for each LED. Each entry has a pattern for the port direction register and a pattern for the line states. A struct is used to set up the data element.

```
typedef struct {  
    uint8_t dir;  
    uint8_t lines;  
} LED_CONFIG;
```

The select[] table is an array of these structs. The table can be stored in FLASH as a 16-bit number to make access easier. The LED pattern is stored in 32 bit unsigned int of which only the first 20 bits are used.

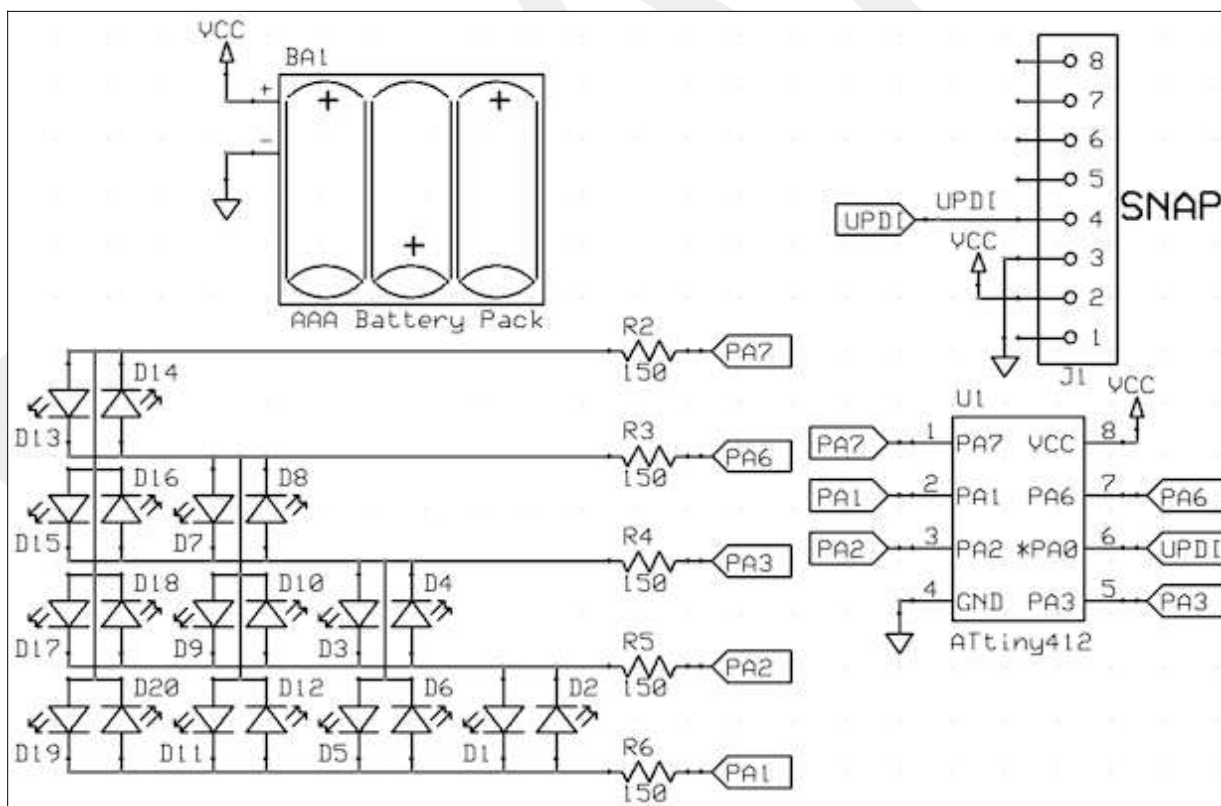


Figure IV-1

2. Remote Temperature Recorder

This example will use the Sleep Controller (SLPCTRL), ADC, USART, and WDT to build a device that measures temperature every 10 minutes and dumps the temperature data on command from the serial port.

3. Interfacing Push Button Switches with debounce

4. Interfacing Single and Multiple 7-Segment LED Displays

This example can also be used to control sixteen segment displays with little modification.

Figure IV_4a

5. HC-SR04 Ultra-sonic Interface for Distance Sensing

6. Make an LED Flicker like a Candle

Random ON time and random OFF time.

7. Interface to a Small DC Motor though a H-Bridge Controller

This example will also show how to control multiple motors. Using TCA for PWM.

8. Interface to Multiple RC Servos using TCA in Split Mode

9. Using the RTC for millis() to free up TCA

All of the tinyAVR® 1-series devices have a Real Time Clock module (RTC). The RTC is clocked by an internal 32.768 kHz low powered oscillator that runs even in Low Power Sleep mode. This example will use the Periodic Interrupt Timer (PIT) section of the RTC to appropriate a 1ms tic for the millis() time function. This frees up the TCA module normally used for the millis() function. If an accurate time base is needed, it can be configured to generate a periodic one second pulse. The hardware configuration in Figure I-1 can be used for demonstration.

The 32.768 kHz clock will bypass the DIV32 divider and use the prescaler set to 1. Writing 32 into the PERIOD field of the RTC.PITCTRLA register will generate a 0.9766 ms period tic every time the PIT counter overflows.

The first step is to configure the clock source for the RTC. Replace init_sysmtime() with this code.

```
// Configure the Clock CLK_RTC
CPU_CCP = CCP_IOREG_gc;           // unlock Change Protected IO Registers
CLKCTRL.OSC32KCTRLA = 0x02;      // Force startup of oscillator.
RTC.CLKSEL = RTC_CLKSEL_INT32K_gc; // 32.768kHz Internal Crystal
while( RTC.PITSTATUS > 0 );       // Wait for PIT to synchronize
```

Then enable the PIT interrupt, set the PERIOD field to 32, and enable the PIT.

```
// Enable PIT interrupt, set PERIOD, and enable PIT
```

```

RTC.PITINTCTRL |= RTC_PI_bm;
RTC.PITCTRLA = RTC_PERIOD_CYC32_gc | RTC_PITEN_bm;
Finally, change the ISR to ISR(RTC_PIT_vect) and reset the PIT flag in it.
ISR(RTC_PIT_vect)
{
    ++totalMilliseconds;
    RTC.PITINTFLAGS = RTC_PI_bm;    // clear the interrupt flag
}

```

The RTC code will be placed into the systime.c,h files since it provides the same millis() functions or it can be kept separate in it's own rtc.c,h files. The io_ctrl.c,h files are copied from previous projects.

An example project using the code discussed is in the ITMC_IV_9_PIT.X.zip project file that can be downloaded from " <https://www.gameactive.org/dist/>".

10. LCD Display interface for HD44780 Displays

This project uses a HD44780 driver to control a 2x24 DMC24277 LCD display. It should work with other HD44780 based LCD displays with minor modification, mostly to the pin number definitions for the interface. It used the 4-bit interface to conserve I/O lines, but could be modified to support an 8-bit interface.

Example code is in the ITMC_IV_10_LCD.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

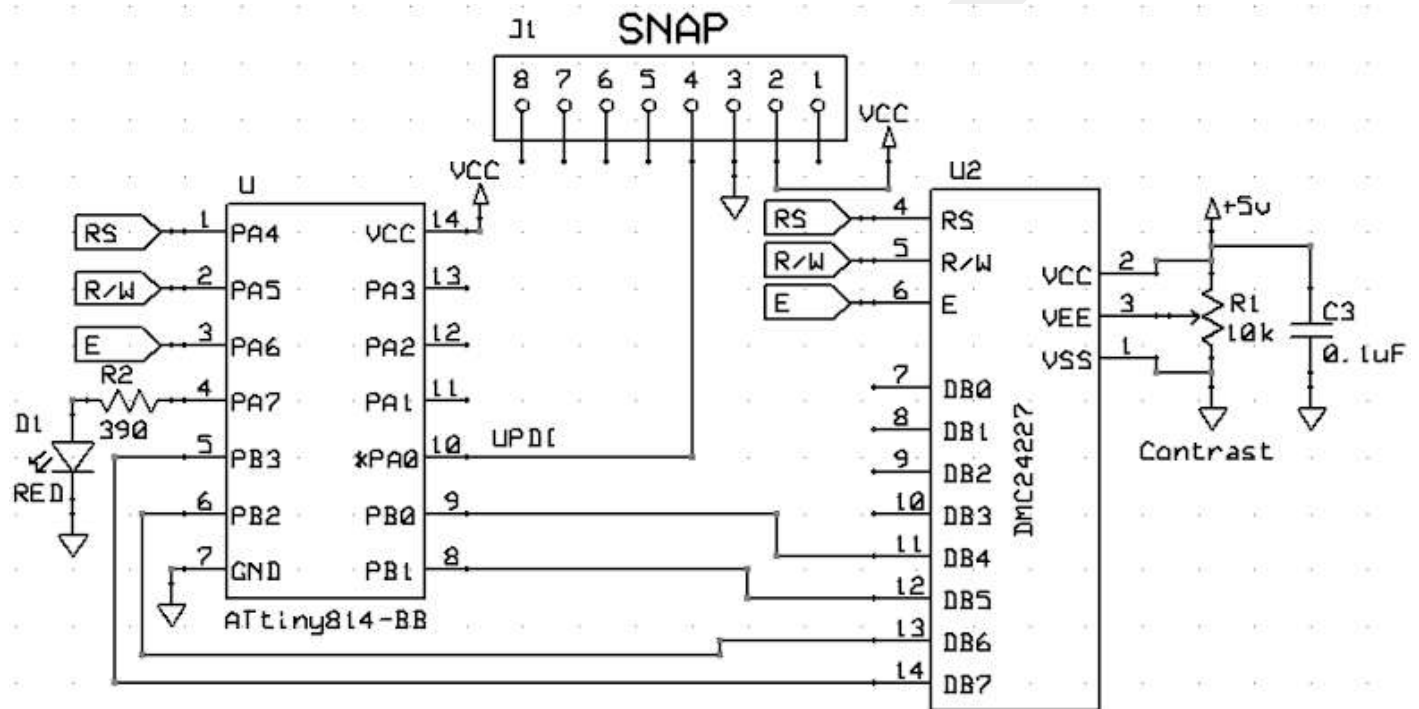


Figure IV 10-1

11. IR Decoder with LCD Display

This project uses an IR receiver module (TSOP38238) from Adafruit and an LCD display from Section IV_10. It will display the Address and Command byte from most 38kHz IR remotes.

Example code is in the ITMC_IV_11_IR_LCD.X.zip project file that can be downloaded from "<https://www.gameactive.org/dist/>".

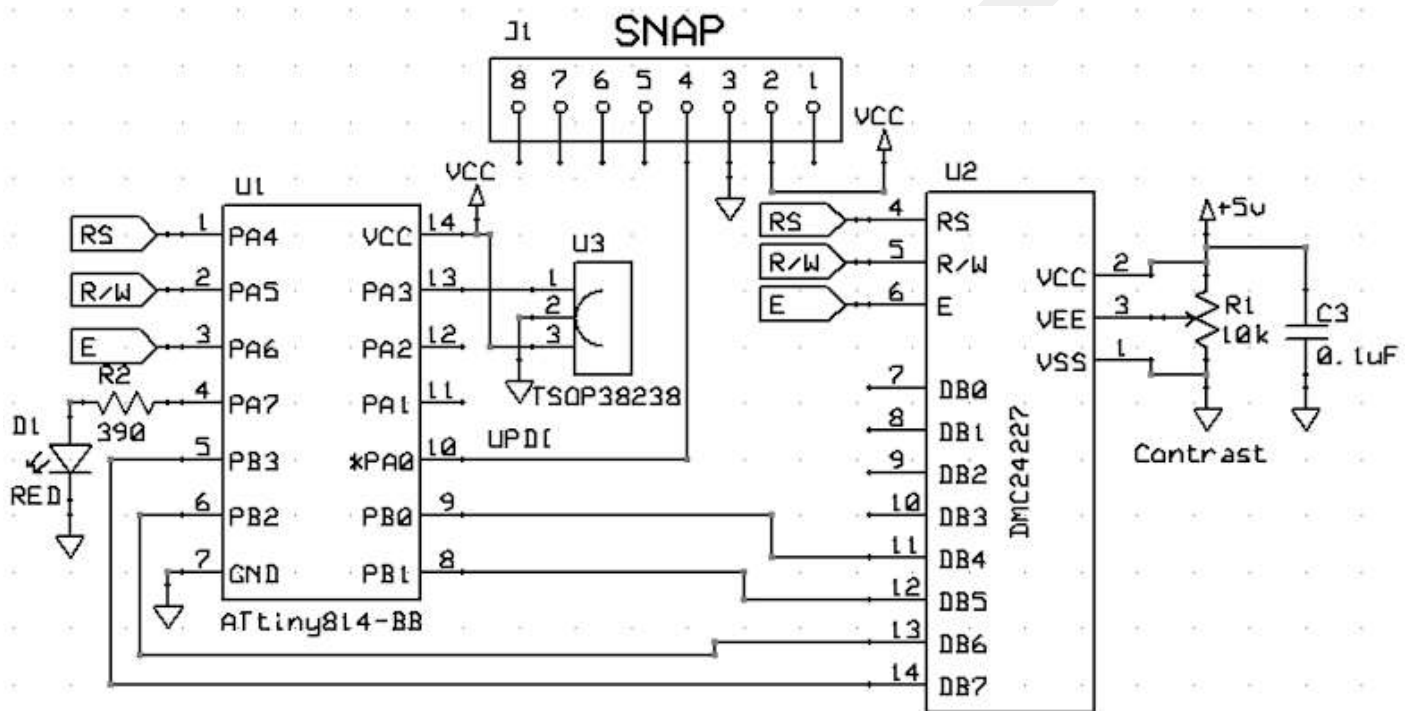


Figure IV 11-1

12. DDS Audio Signal Synthesis – DAC and PWM

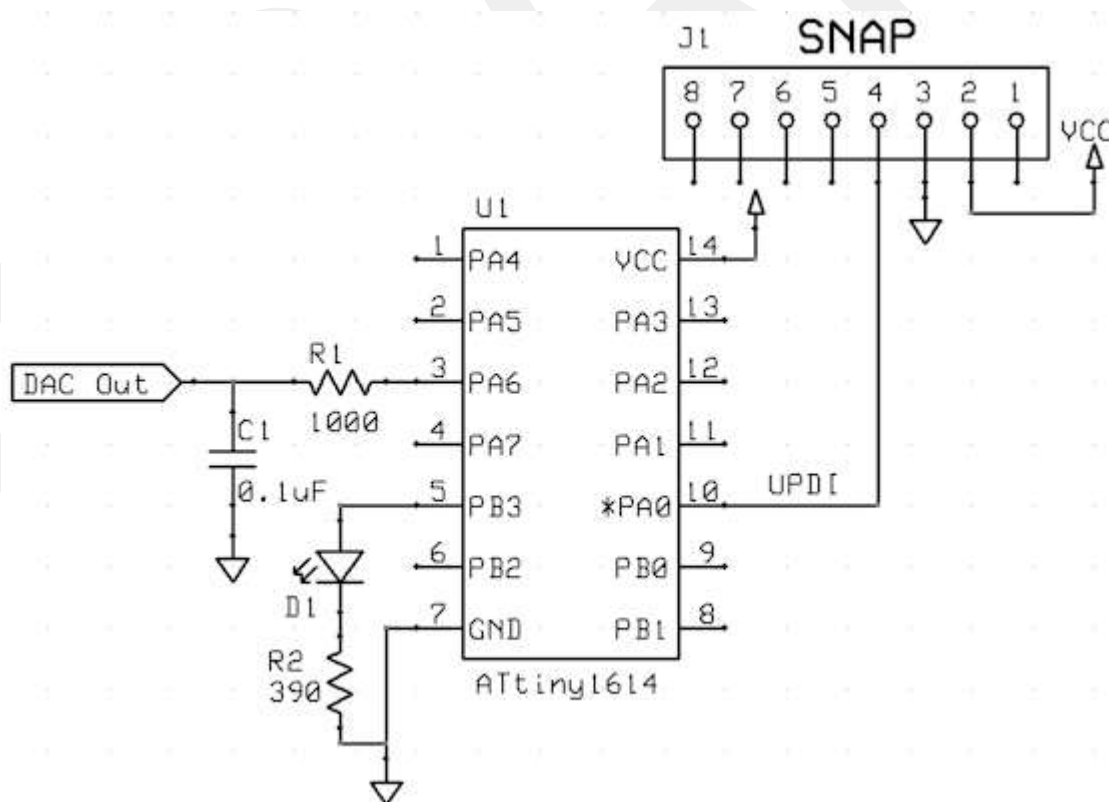
This project provides two ways to generate audio signals using Direct Digital Synthesis (DDS). The first takes advantage of the AVR series devices providing a DAC. This generates a very clean signal and requires very little filtering. The second method uses the TCA timer/counter module to generate three separate PWM channels. For best performance, these channels need a sharp (4-pole) filter to convert the PWM signal into a smooth waveform.

The basic operation of DDS is to use a fixed waveform table and step through it at different increments to generate the same waveform at different frequencies. If a 16-bit accumulator is used and it is incremented by 256 each sample period, the upper 8-bits can be used to index a 256 element waveform table. Adding 256 will increment the upper byte by 1 each time. To generate a waveform of twice the frequency, add 512 each time. To generate a frequency between the base and twice the base, use values between 256 and 512. Value greater than 512 will generate even higher frequencies. A value of 256×64 is about the upper bound as this would result in only 4 samples per period.

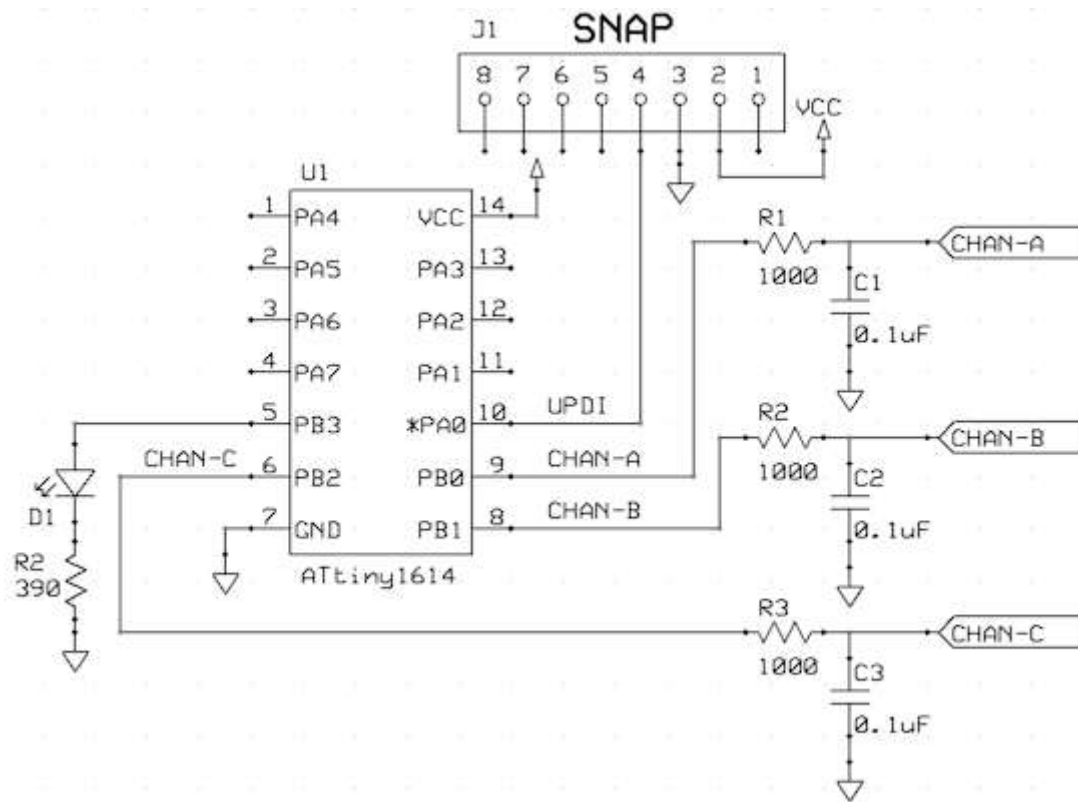
TCA is set to overflow every 12.8us. The 16-bit (non-split) mode is use to allow experimenting with longer PWM registers with little modification. The base frequency for a 256 sample table at 12.8us per sample is 305.2 Hz. The system clock is set to 20 MHz to get maximum performance.

NOTE: The PWM COMP value has to be greater than 40 or so to allow the ISR to update it before the the count exceeds its new value being loaded.

Example code is in the ITMC_IV_12_Audio_DAC.X.zip and ITMC_IV_12_Audio_PWM3.X.zip project files that can be downloaded from "<https://www.gameactive.org/dist/>".



DAC version
Figure IV 12-1



PWM version
Figure IV 12-2

SECTION V - Power

Needs of the project. MCU 1.8vdc - 5.5vdc. External devices; LEDs, motors, servos, speakers

Voltage, peak current, capacity (mAh)

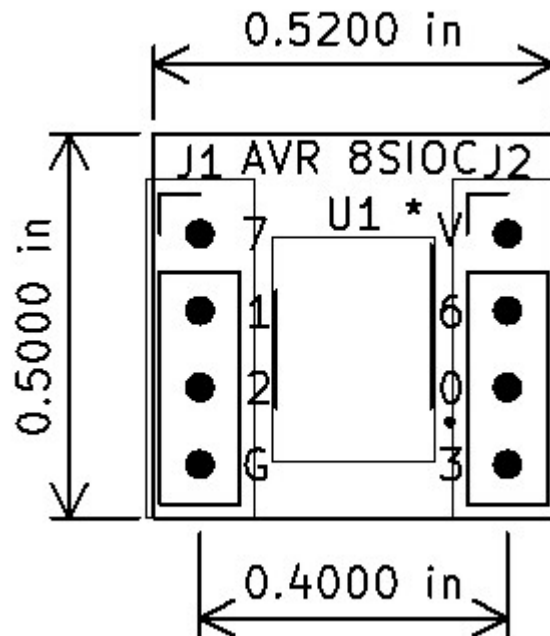
Batteries - 9v, AA, AAA, LiPo, coin cell.

Wall Plug - 2.1mm female connector.

APPENDIX I - BOARD LAYOUTS

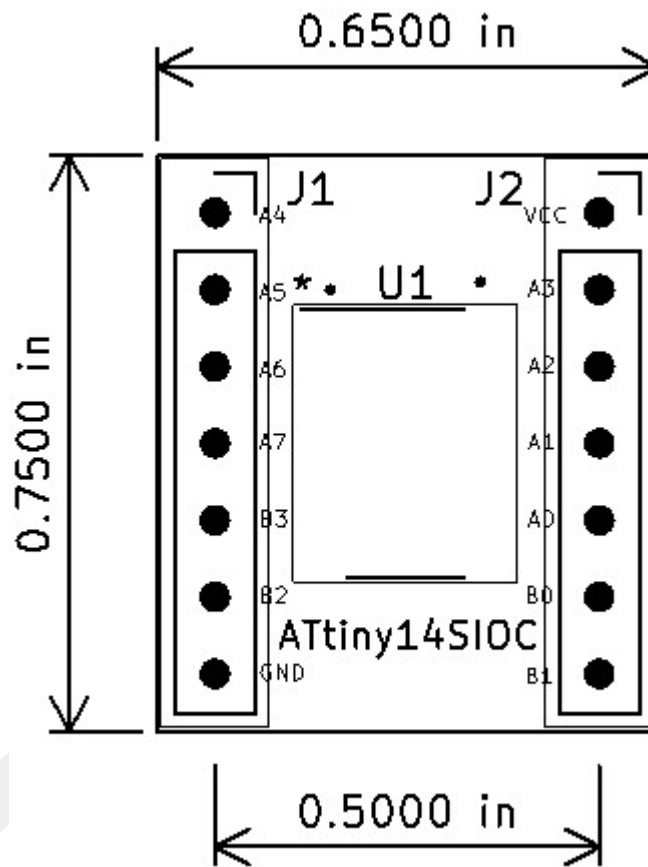
A. ATtiny8SIOC

For ATtiny212 and ATtiny412 devices.



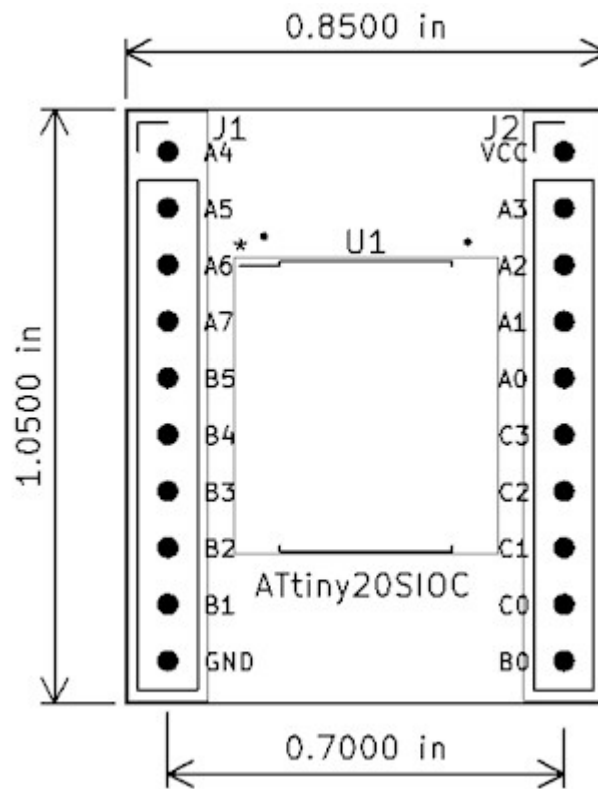
B. ATtiny14SIOC

For ATtiny814 and ATtiny1614 devices.



C. ATtiny20SIOC

For ATtiny1616 and ATtiny3216 devices.



APPENDIX III – Schematics

A. ATtiny8SIOC

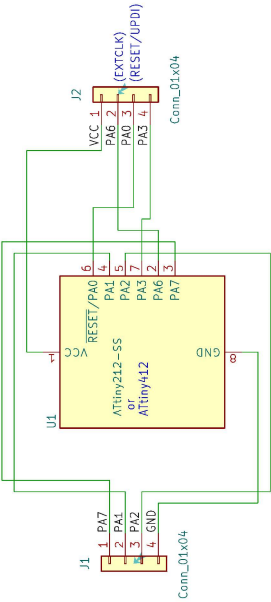


Table 5-2. PORT Function Multiplexing, Eight Pins

SOIC 8-Pin	Pin Name (1,2)	Other/ Special	ADC0	AC0	DAC0	USART0	SPI0	TWI0	TCA0	TCB0	TCD0	CCL
6	PA0	RESET/UPDI	AIN0			XDIR	SS					LUT0-IN0
4	PA1		AIN1			TxD(3)	MOSI	SDA	WO1			LUT0-IN1
5	PA2	EVOUT0	AIN2			RxD(3)	MISO	SCL	WO2			LUT0-IN2
7	PA3	EXTCLK	AIN3	OUT		XCK	SCK		WO0/WO3			
8	GND											
1	VDD											
2	PA6		AIN6	AIN0	OUT	TxD	MOSI(3)			WO0	WOA	LUT0-OUT
3	PA7		AIN7	AINP0		RxD	MISO(3)		WO0(3)		WOB	LUT1-OUT

For ATtiny2:2 or ATtiny4:2

Sheet: /
File: ATtiny8SIOC.kicad_sch

Title: **ATtiny 8SIOC**

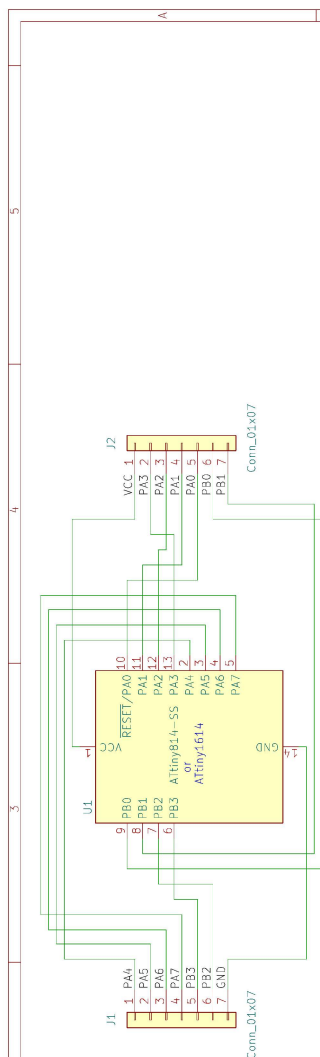
Size: USLetter | Date: 2024-07-27

KiCad E.D.A. | KiCad 7.0.10

Rev:

Id: 1/1

B. ATtiny14SIOC



5.1 Multiplexed Signals

Table 5-1. PORT Function Multiplexing

[illegible]

ATTiny1614 has a few additional multifunction pin options.

ATtiny814 or ATtiny1614

Sheet: /
File: ATTiny14SIOC.kicad_sch

Title: *Attiny 14SIOC*

Size: USLetter	Date: 2024-07-27
KiCad E.D.A. kicad 7.0.10	

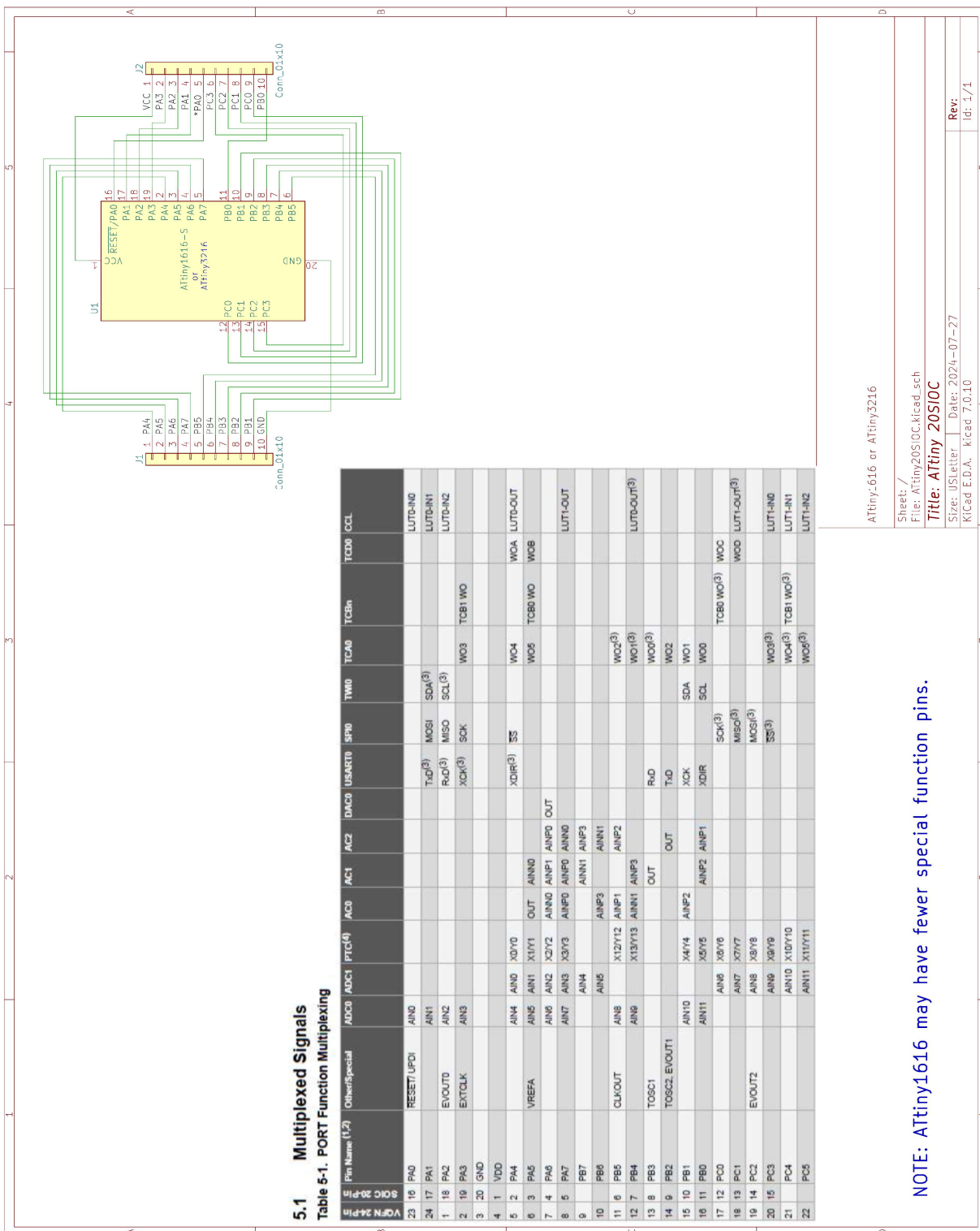
C. ATtiny20SI0C

5.1 Multiplexed Signals

Table 5-1. PORT Function Multiplexing

[illegible]

NOTE: ATtiny1616 may have fewer special function pins.



APPENDIX IV - UPDI Programmers

<https://www.microchip.com/en-us/development-tool/ATATMEL-ICE>

- Atmel-ICE \$203.65

<https://www.microchip.com/en-us/development-tool/PG164100>

- MPLAB® Snap In-Circuit Debugger/Programmer \$39.20 (Recommended)

https://ww1.microchip.com/downloads/en/DeviceDoc/ETN36_MPLAB%20Snap%20AVR%20Interface%20Modification.pdf

- ETN-36 MPLAB Snap AVR Interface Modification. Needed to program AVR devices.

- Add a 2.2k ohm resistor between VCC and UPDI pins as recommended in ETN-36.

<https://www.microchip.com/en-us/development-tool/ev50j96a>

- ATTINY3217 CURIOSITY NANO EVALUATION KIT includes a UPDI programmer and debugger. \$14.99

See <https://forums.adafruit.com/viewtopic.php?t=211365> on how to use these with MPLAB.

<https://www.adafruit.com/product/5893>

- Adafruit High Voltage UPDI Friend - USB Serial UPDI Programmer \$9.95

<https://www.adafruit.com/product/5879>

- Adafruit UPDI Friend - USB Serial UPDI Programmer \$6.95

References

ref1: <https://www.microchip.com/en-us/products/microcontrollers-and-microprocessors/8-bit-mcus/avr-mcus>

ref2: ATtiny212-214-412-414-416-DataSheet-DS40002287A.pdf

ref3: <https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide>

ref4: <https://www.microchip.com/en-us/development-tool/PG164100> - SNAP Debugger/Programmer

ref5: ATtiny417-814-816-817-DataSheet-DS40002288A.pdf

ref6: ATtiny1616_40001893B.pdf

APPENDIX V - Resources

ATtiny8SIOC.pcb ordered 01aug24, 3 for \$1.30

ATtiny14SIOC.pcb ordered 01aug24, 3 for \$2.40

ATtiny20SIOC.pcb ordered 03aug24, 3 for \$4.45

(2)SNAP ordered 01aug24, 2 for \$89.64 (see AVR mod sheet)